Lab 5: ADC Module and Serial Interface to MATLAB

Engineering Physics 3BB3 Embedding and Programming a Micro-controller

Lab Section: L01-Wednesday
Instructor's Name: Dr. Adriaan Buijis

Written by: Ethan Chan

Date Lab Performed: February 11, 2026
Date of Submission: February 27, 2026

Abstract

Lab 5 ADC Module and Serial Interface to MATLAB details the design, and profiling of a mixed signal embedded data acquisition functioning as a "poor-man's USB oscilloscope". Using the Texas Instruments MSP430 (MSP) microcontroller's integrated 10-bit Analog-to-digital Converter (ADC10) flashed with C code using Code Composer Studio (CCS), the system samples time varying waveforms, and transmitted data via Universal Asynchronous Transmission (UART) to display the output on a Graphical User Interface (GUI) made with MATLAB APP Designer. A frequency analysis was performed comparing the MATLAB output to a commercial digital storage oscilloscope, demonstrating an effective functional bandwidth from 816 Hz up to the 163 kHz Nyquist threshold, beyond which distinct beat phenomena, and aliasing were observed. Bottleneck cycle profiling via C code revealed that the ADC10 achieved rapid burst sampling at 49 cycles per sample establishing a sample time of $3.06 \pm 0.06\ \mu s$, while UART profiling confirmed a 19-cycle CPU block for buffer loading. Though this CPU overhead is minimal, the physical 115200 baud transmission ultimately creates a significant 35 ms data-blind spot. An uncertainty analysis was conducted on the time domain, defining an absolute temporal measurement confidence interval for the localized burst sampling.

1.Introduction

The objective of this lab is to use the ADC10 module of the MSP430G2553 microcontroller to design and build a USB oscilloscope. Analog-to-Digital Converters (ADCs) are fundamental components in analog and digital electronics, allowing continuous-time physical signals to be processed by discrete-time computational systems.

The experiment involved configuring the microcontroller to sequentially sample an external waveform, truncate the 10-bit data to 8 bits for efficient serial packaging, and transmit it to a host PC using UART. A custom MATLAB App Designer interface was developed to plot the received data in real-time, utilizing a software-defined trigger to stabilize the waveform. The lab demonstrated real world data acquisition constraints, heavily emphasizing the trade-offs between sampling frequency, serial communication bottlenecks, hardware polling cycle overhead, and the inherent limits of data transmission. An extensive frequency sweep was also utilized to empirically verify the onset of experimental aliasing and the beat phenomenon near the theoretical bandwidth limits, while direct Timer_A profiling, and transmission calculations quantified the exact cycle costs of analog conversion, and UART transmission blind spot.

2. Theory

2.1 The ADC10 Module and SAR Architecture [2]

The ADC10 module embedded on the MSP operates on a 10-bit Successive Approximation Register (SAR) capable of 200 kilo-samples per second (ksps). A 10- bit ADC bins the measurements range into 2^{10} between discrete digital levels, ranging from 0 to 1023. The sample phase, where the internal capacitor charges to match the external analogue voltage during sample and hold phases. Once the sampling window closes, the charge is held, and the SAR state machine uses a binary search algorithm, iteratively comparing the held voltage against an internal to digital to analog converter (DAC) to resolve the final digital code.

For this lab code the sample and hold phase was configured to last for 4 clock cycles using the (ADC10SHT_0) control register bit. Because the actual binary conversion phase is hardware fixed to 13 clock cycles to allow the comparator to settle, this establishes a strict hardware minimum of 16 clock cycles per single analog conversion.

The numeric integer value register by the ADC, N_{ADC} is calculated based on the input voltage V_{IN} and the reference voltages V_{R+}, and V_{R-}:

$$N_{ADC} = 1023 \times \frac{V_{IN} - V_{R-}}{V_{R+} - V_{R-}} \tag{1}$$

For this lab, internal reference was configured such that V_{R+} is connected to V_{CC} a nominal 3.3 V and V_{R-} is connected ground (0 V). Rearranging the formula allows the analog voltage to be reconstructed in the software:

$$V_{IN} = \frac{N_{ADC}}{1023}\,(V_{R+} - V_{R-}) + V_{R-} \tag{2}$$

2.2 Nyquist Shannon Theorem [3]

The fidelity of the digital waveform depends strictly on the sampling frequency f_s. The Nyquist Shannon sampling theorem states that a continuous time signal can be perfectly reconstructed if it is sampled a rate strictly greater than twice its maximum frequency component f_{max}:

$$f_s > 2f_{max} \tag{3}$$

If the signal is sampled below the Nyquist rate, high frequencies components are “folded” back onto lower frequencies, creating a distorted, in accurate data known as aliasing.

3. Methods

The tasks Lab 5 ADC Module and Serial Interface to MATLAB were the following [1]:

Task #3.1.1: Download the C code template and write a program in CCStudio to enable the MCU to digitize an analog input signal using the ADC10MEM register. Additionally, it should allow the MCU to transmit the sampled data.

Task #3.1.2: Write a program in MATLAB to receive and graph the data. Modify your MATLAB code such that it shows the yscale in correct voltage units. If you still have time left, enhance your MATLAB code such that the displayed waveform is synchronized to a rising or falling edge of the wave form.

Task #3.1.3: Discuss the advantages and limitations of your system as a viable real-time waveform digitizer (or oscilloscope).

3.1 Hardware Assembly and Safety Verification

The circuit was built and tested using a breadboard and the MSP-EXP430G2ET LaunchPad development kit. To enable hardware UART communication between the MSP430G2553 and the host PC, the RXD, and TXD jumpers on were removed, rotated 90 degrees, then placed horizontally. A red jumper wire was connected to P1.4 to serve as the analog input probe, and a black common ground wire was established.

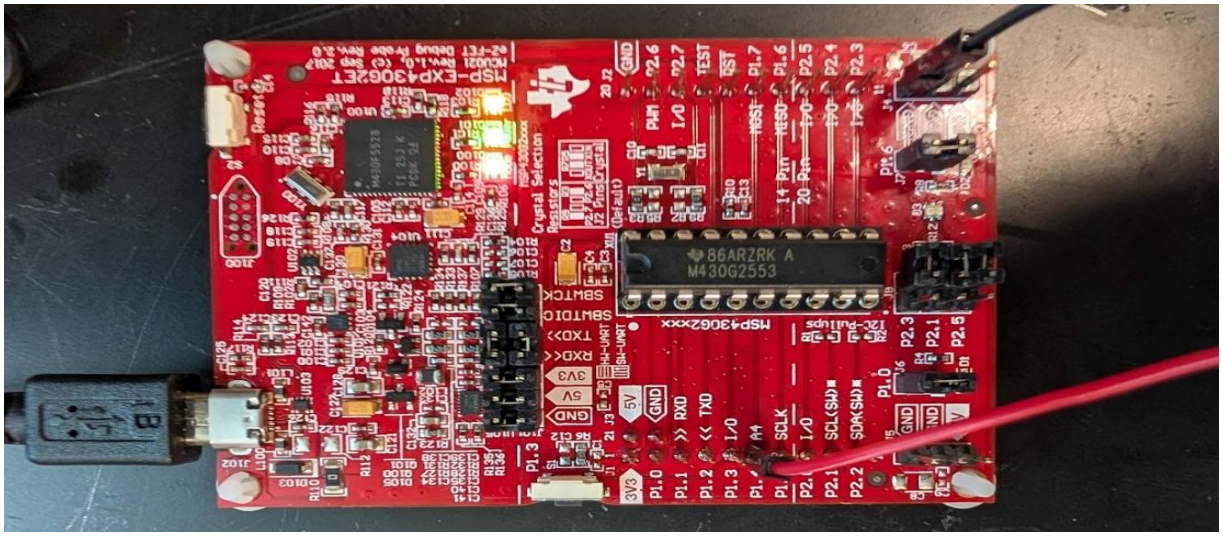

Figure 1:Hardware Assembly to enable UART RXD, and TXD jumper blocks were rotated 90 degrees. Red jumper wire to P1.4 as an analogue input probe, and a black ground wire to GND pin.

Signal Safety Verification: The microcontroller operates on a 0 to 3.3 V logic level. A laboratory function generator was turned on while physically disconnected from the microcontroller. A pure sine wave was generated and configured with a 1.5 Vpp peak-to-peak amplitude and a 1.0 Vdc positive DC offset.

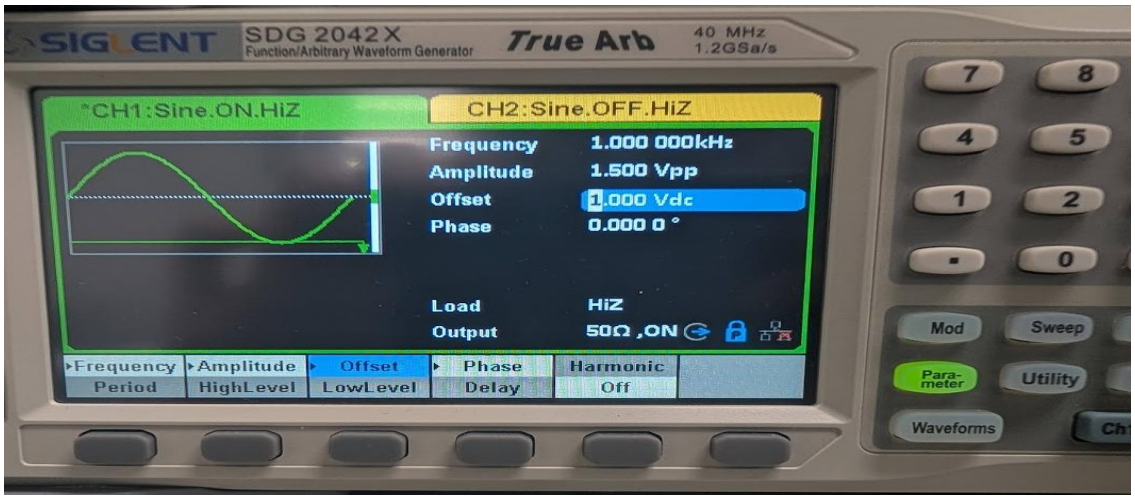

Figure 2: Signal Safety Verification ensuring 0 to 3.3V logic levels, a sine wave 1.5 Vpp and 1.0 Vdc was generated

Oscilloscope Pre-Testing: Before connecting to the Launchpad, a Silent digital storage oscilloscope was attached directly to the function generator. The oscilloscope trace was visually inspected to confirm the signal strictly oscillated between 250 mV and 1.750 V, ensuring no negative voltages or spikes could damage the ADC10 input.

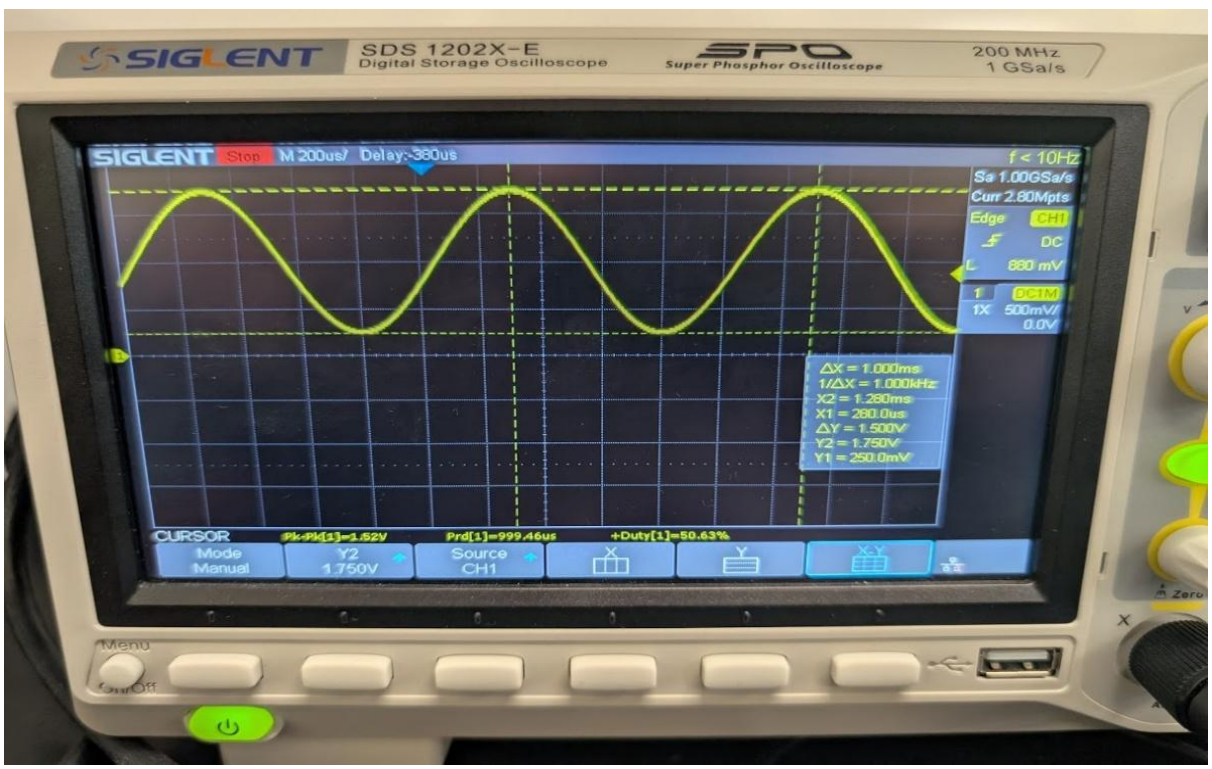

Figure 3: Oscilloscope Pre-Testing confirmed sine wave oscillated between 250 mV, and 1.750 V.

Once verified, the function generator's ground lead was connected to the Launchpad ground, and the active signal lead was connected to the P1.4 analog input pin.

Firmware and Software Execution: The C-code was flashed to the microcontroller via Code Composer Studio. The code configures the ADC10 to perform multiple sequence conversions and stores 400 points in an array before transmitting them sequentially over UART at 115200 baud. The MATLAB App was launched, the correct COM port was identified, and the "Run" button was pressed to initiate the trigger and visualize the serial data.

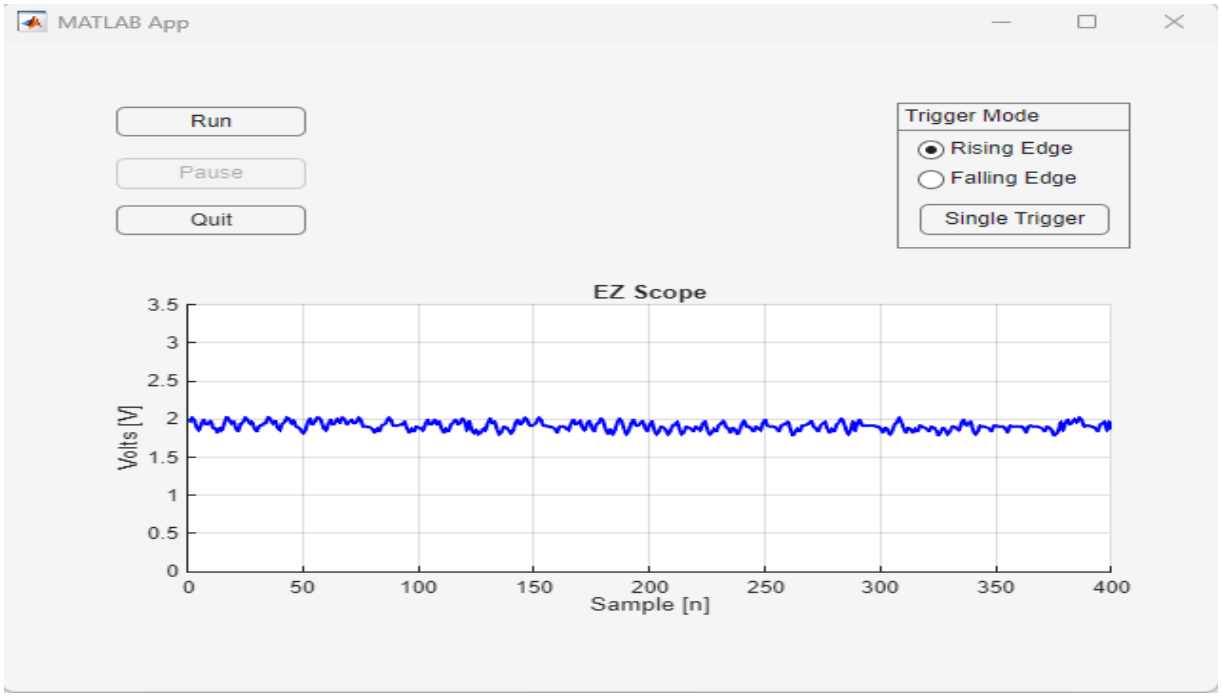

Figure 4: First firmware and software execution viewed on EZ Scope Custom MATLAB APP.

Note: that in the first iteration of the EZ scope the X-axis samples [n] as placeholder

3.2 Firmware Architecture and Step-by-Step Execution

The microcontroller firmware was designed with a highly deterministic, interrupt-driven architecture to facilitate communication between the ADC and UART modules without data loss. **The Firmware C-Code (MSP430) can be found in Appendix C: System Codebase**

The step-by-step logic executes as follows:

1. **Initialization (Init functions):** The main() routine first disables the watchdog timer to prevent timeout resets during serial transmission. The system clock is calibrated to exactly 16 MHz.

2. **UART Configuration (Init_UART):** The UART module is configured to communicate at 115200 baud. To achieve this, a clock division calculation is performed: $\frac{16{,}000{,}000\ Hz}{115200\ baud} \approx$ 138.88. The integer portion, 138, is loaded into the UCA0BR0 prescaler register, and the fractional remainder is handled by setting the modulation register UCA0MCTL to UCBRS_7 to minimize transmission errors.

3. **ADC Configuration (Init_ADC):** The ADC is set up for Channel 4 (INCH_4) with a 4-cycle sample-hold time (ADC10SHT_0). The CONSEQ_2 command places the ADC in "Repeat single channel" mode, allowing for rapid burst sampling.
4. **Sleep and Synchronization:** The main loop enables the receive interrupt (UCA0RXIE) and places the CPU in Low Power Mode 0 (LPM0). The system sits entirely idle until MATLAB initiates communication.
5. **RX Interrupt (USCI0RX_ISR):** When MATLAB sends the synchronization character 'U', this interrupt fires. It reads the buffer, sets the task_ready flag to 1, and wakes the CPU from LPM0.
6. **Data Acquisition (ADC10_ISR):** Woken up, the main loop enables the ADC interrupt. The code enters a blocking for loop, waiting for the hardware flag (ADC10IFG) to confirm a sample is ready. The 10-bit result (0-1023) is bit-shifted right by 2 (ADC10MEM >> 2), mathematically truncating it to an 8-bit value (0-255) to fit inside a single UART character frame. This is stored in a 400-point array.
7. **TX Interrupt (USCI0TX_ISR):** Once all 400 samples are acquired, the ADC interrupt is disabled, and the transmit interrupt (UCA0TXIE) is triggered. This interrupt sequentially offloads the entire array to the UART transmit buffer (UCA0TXBUF), sending the data block to MATLAB.

3.3 Host Software Integration and Logic

The MATLAB App Designer interface manages the host-side logic.

The MATLAB Application Designer Code can be found in Appendix C: System Codebase

The execution relies on the following key processes:

1. **Triggering the MCU:** When the "Run" button is pressed, the code executes write(app.comport, 'U', "char"); to ping the MCU's USCI0RX_ISR.
2. **Data Reconstruction:** MATLAB executes a blocking read to collect the 400 bytes. The 8-bit array is converted back to an analog voltage using the equation: voltage_data = (double(raw_data)./ 255) * 3.3.
3. **Software Trigger Alignment:** To stabilize the scrolling visualization, a software trigger function performs a linear loop search through the incoming 400-point voltage array, evaluating sequential val_index and val_next pairs. Depending on the UI selection, it checks if the waveform crosses a defined threshold (e.g., 1.5 V) in a rising or falling direction. Once the exact crossing index is found, the data is sliced starting from that index, forcing the plot to always anchor the visualization at the same phase angle.

Below are the is the first implementation of the EZ Scope of the rising and falling edge of the 1kHz 1.5 Vpp 1 Vdc sine wave from the signal generator.

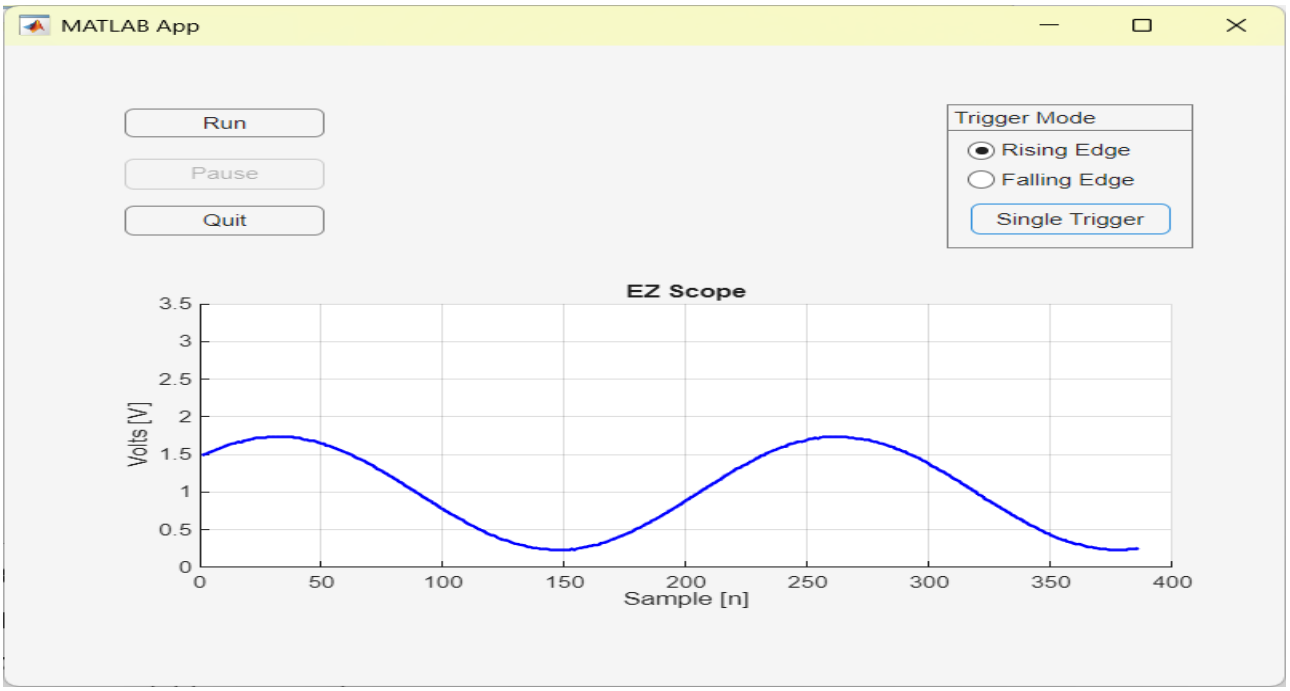

Figure 5: EZ Scope of the rising edge of the 1kHz 1.5 V pp 1 Vdc sine wave from the signal generator.

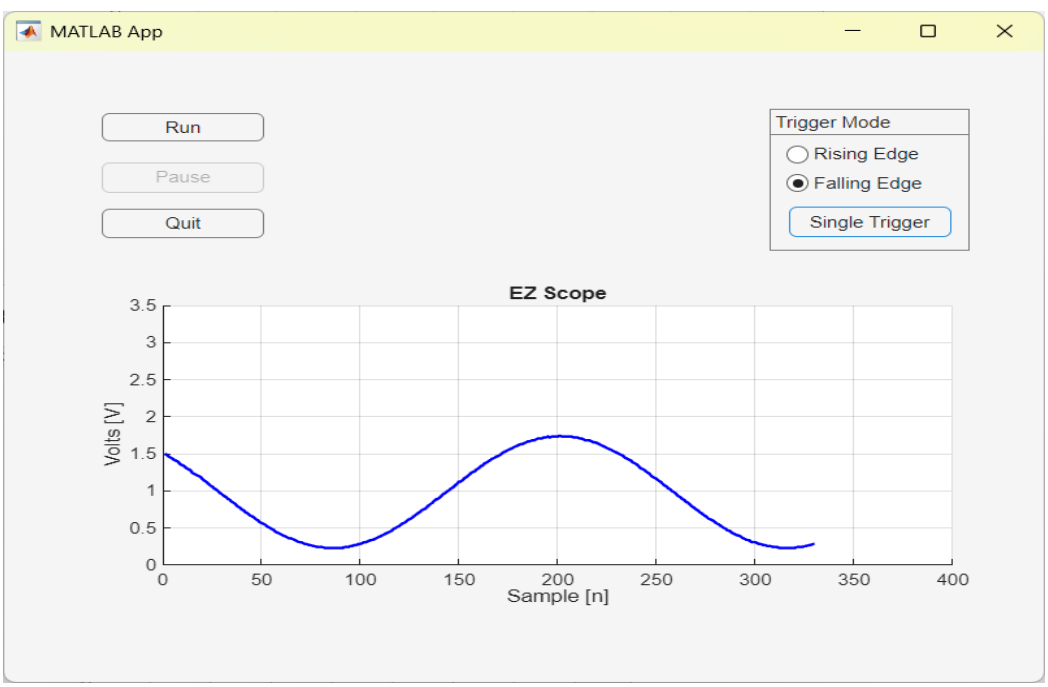

Figure 6: EZ Scope of the falling edge of the 1kHz 1.5 V pp 1 Vdc sine wave from the signal generator

4 Results

4.1 System Profiling and X-Axis Time Conversion

Upon injecting the verified 1kHz 1.0 Vpp 1.5 Vdc sine wave into P1.4, the MATLAB interface accurately displayed the waveform. The software trigger algorithm successfully stabilized the trace by aligning the plot to the user-defined edge threshold.

To quantify system bottlenecks and establish accurate timing, cycle profiling was conducted using dedicated C code leveraging the internal Timer_A module.

The Cycle Profiling Code can be found in Appendix C: System Codebase

While the hardware minimum is 16 cycles, measuring the actual implementation revealed that the entire ADC conversion sequence including the necessary interrupt servicing, while-loop polling, and bit-shifting operations strictly consumed 49 clock cycles per sample.

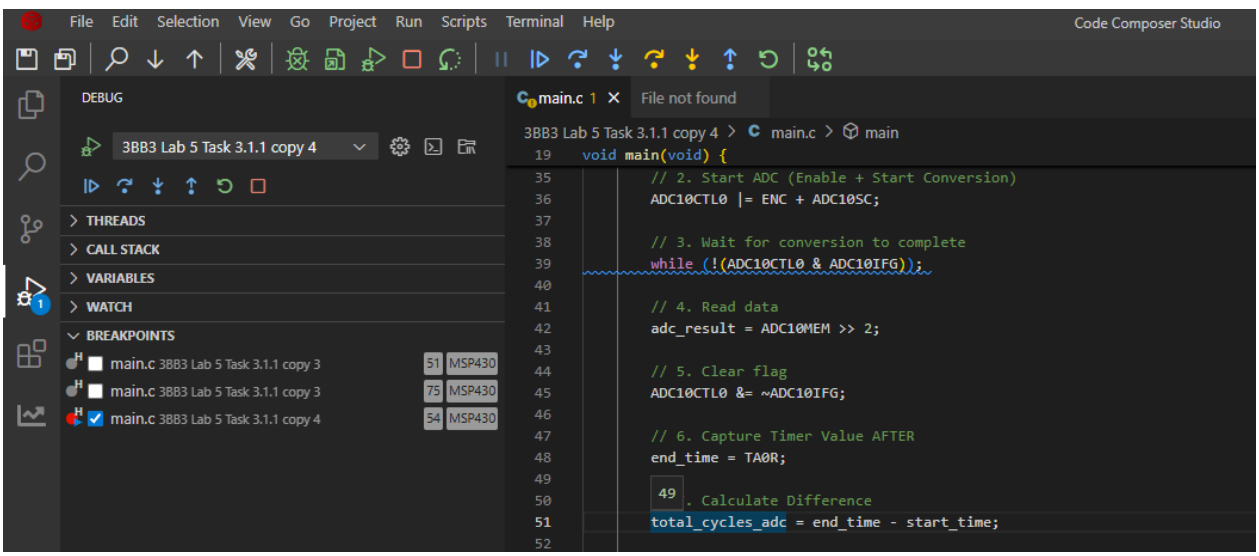

Figure 7 Cycle Profiling Code showing entire ADC conversion sequence with interrupt service, while looping polling, and bit shifting operations consuming 49 clock cycles per sample.

Operating at the defined 16 MHz system clock, this yields a sampling time per point.

$$T_{sample} = \frac{Cycles}{Clock\ Frequency} = \frac{49}{16{,}000{,}000\ Hz} = 3.0625\ \mu s \tag{4}$$

Resulting in $3.0625\ \mu s$ per sample, resulting in a localized burst sampling rate of $326\ ksps$.

Furthermore, additional cycle profiling on the UART transmit subroutine determined that writing, and loading a single byte to the UCA0TXBUF register consumed exactly 19 clock cycles.

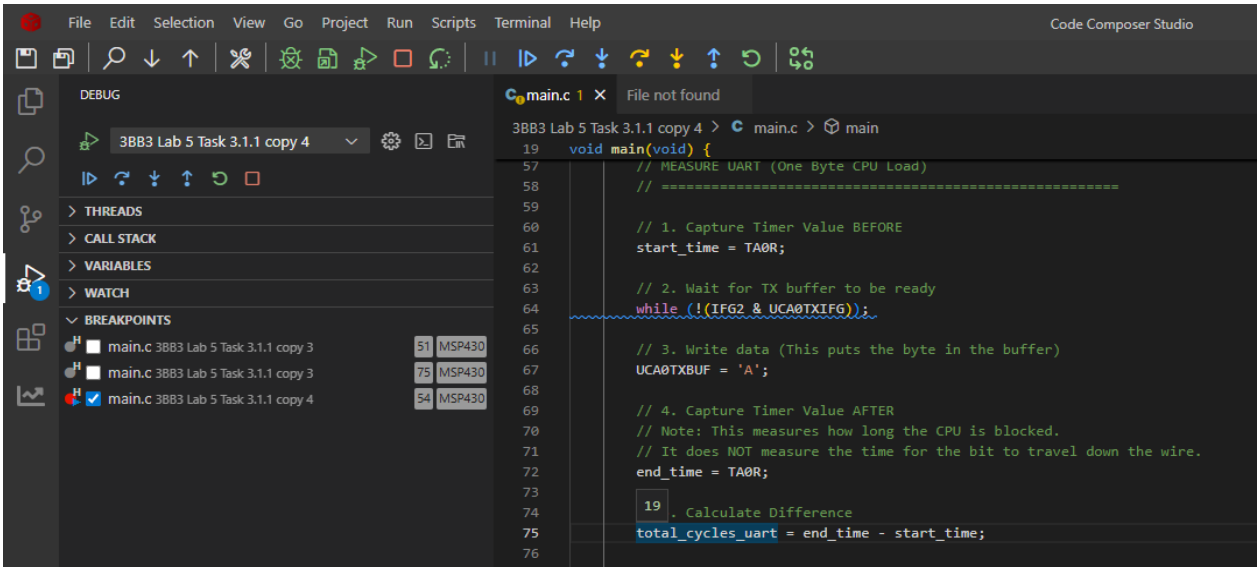

Figure 8: Cycle Profiling Code showing entire UART subroutine to load a single byte to the UCA0TXBUF register consumed 19 clock cycles.

$$T_{Load\ Buffer} = \frac{Cycles}{Clock\ Frequency} = \frac{19}{16{,}000{,}000\ Hz} = 1.1875\ \mu s \tag{5}$$

However, this brief CPU blocking time only accounts for the internal memory transfer; it does not measure the actual physical time required for the bits to travel down the wire.

To determine the true transmission blind spot, the physical baud rate must be calculated. The UART module is configured to 115200 baud, which equates to exactly 115,200 bits per second. The standard UART protocol requires 10 bits per byte transmitted (1 start bit, 8 data bits, and 1 stop bit). Therefore, transmitting the entire 400-byte data array requires sending 4,000 bits. Calculating the total transmission time yields:

$$Transmission\ Time = \frac{4000\ bits}{115200\ \frac{bits}{seconds}} = 0.0347\ seconds \tag{6}$$

This physical transmission inherently takes roughly 35 ms.

After finding out that each sample takes precisely 49 cycles, this deterministic timing was applied to upgrade the MATLAB software. Instead of plotting the X-axis against an arbitrary sample index [n], the X-axis was changed to display true time.

By multiplying sample of 3.0625 μs per sample by the 400 samples highly accurate vector was generated.

$$T_{window} = T_{sample} \times N_{points} = 3.0625\mu s \times 400 = 1{,}225\ \mu s = 1.225\ ms \tag{7}$$

Thus, converting the interface into a time-domain oscilloscope, making the whole window time 1,225 μs in future versions of the EZSCOPE App. Below is the 6th version of the EZSCOPE with a voltage noise measurement displaying the whole window time in the X-axis.

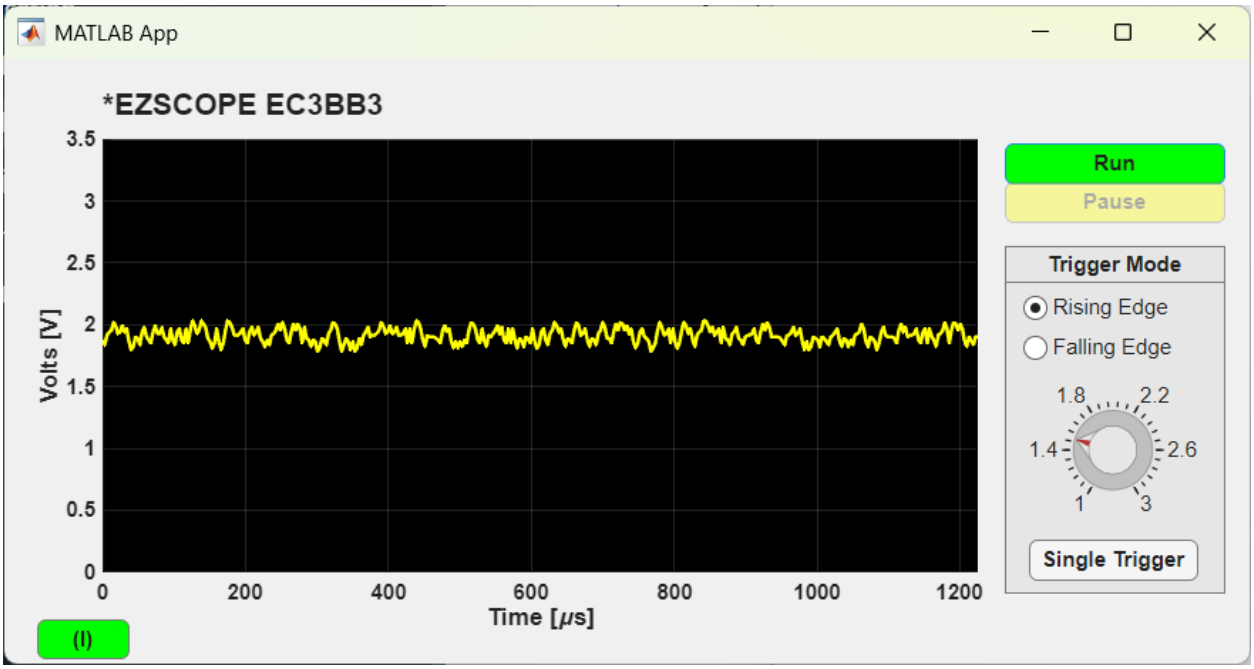

Figure 9:6th version of the EZSCOPE with a voltage noise measurement displaying the whole window time in the X-axis.

4.2 Frequency Analysis and Oscilloscope Comparison

To study the frequency-dependent limitations of the system, a frequency sweep was performed on the function generator maintaining the 1.5 Vpp, 1.0 Vdc settings. The external Siglent oscilloscope was kept in parallel with the P1.4 input to serve as a benchmark.

The frequency sweep revealed that the effective functional bandwidth of the system spans from approximately 800 Hz up to the Nyquist rate of 163 kHz.

The lower bound of minimum visible frequency (f_{min}) is:

$$f_{min} = \frac{1}{T_{window}} = \frac{1}{0.001225\ s} \approx 816.3\ Hz \tag{6}$$

~800 Hz is dictated by the 400-point array limit, and the time per sample of 3.06 μs.T he total acquisition window is $400 \times 3.06\ \mu s \approx 1.22\ ms$, any signal with a period longer than 1.22 ms (i.e., frequencies below 816 Hz) cannot fit a full cycle onto the screen.

Using the trigger function 816 Hz is on the EZSCOPE, two peaks can be seen, so a period is repeated but the end of the sample cannot be seen lower to ~1V.

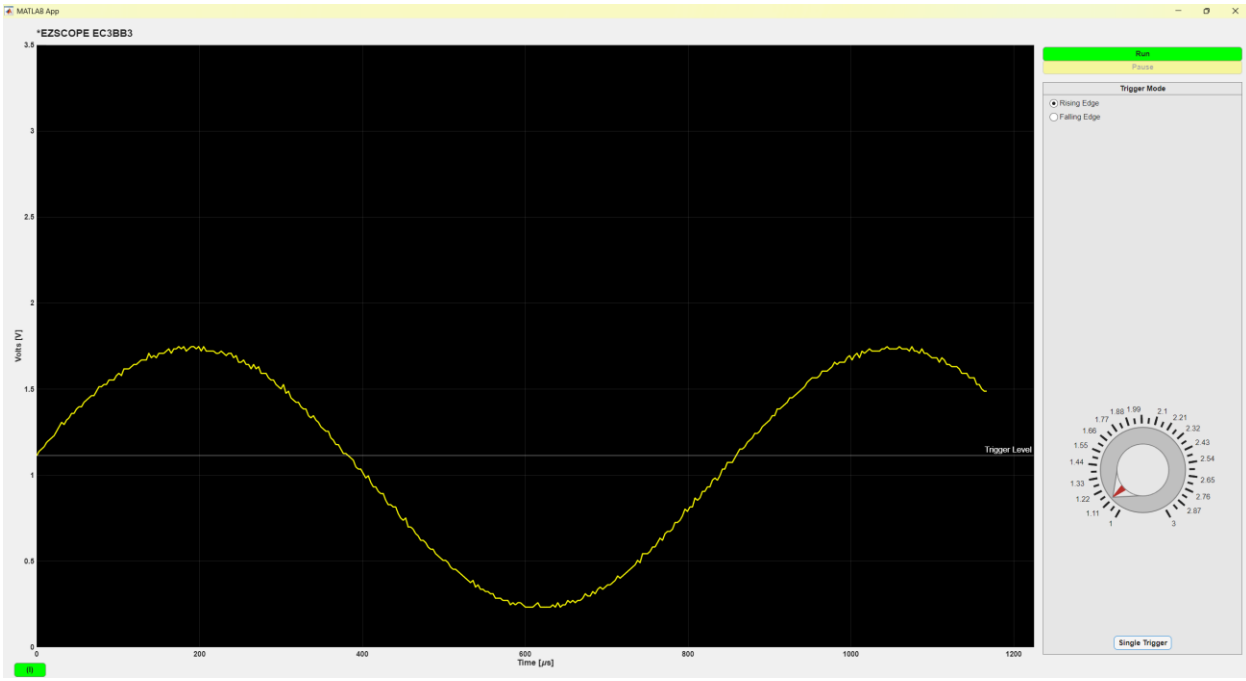

Figure 10: Trigger function captures 816 Hz Sine wave on the EZSCOPE which has its period repeat.

Interestingly lowering the frequency to 617.3 Hz, using the rising edge trigger function the sample shows one peak, and a period is nearly captured.

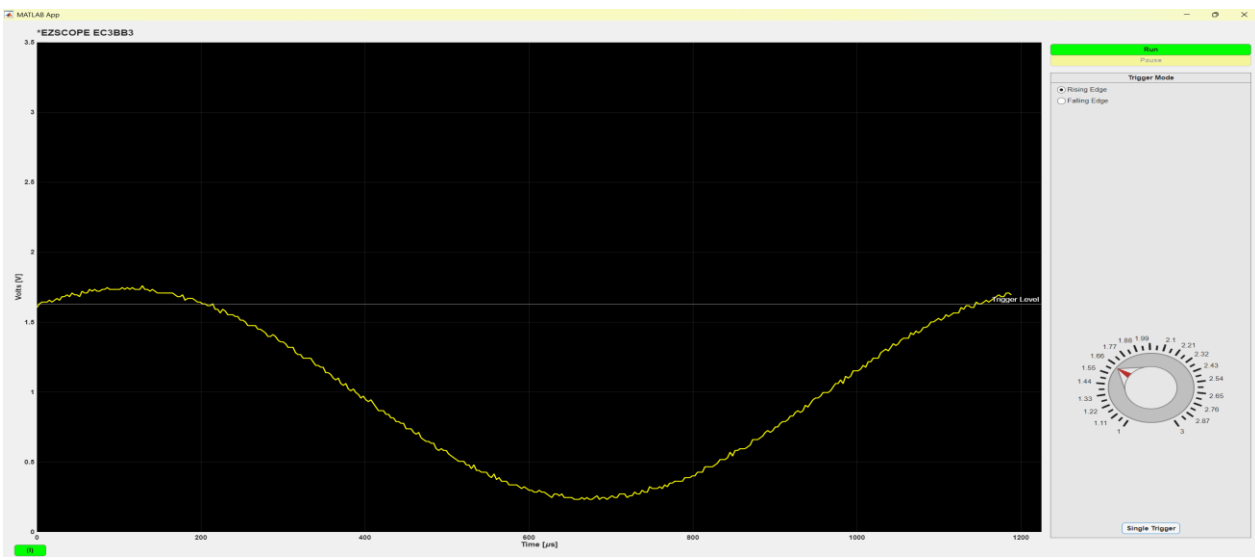

Figure 11: Trigger function captures 617.3 Hz Sine wave on the EZSCOPE the sample almost captures one period.

As the input frequency approached the theoretical Nyquist limit of the localized sampling loop Nyquist Limit ($f_{Nyquist}$):

$$f_S = \frac{1}{T_{sample}} \approx 326{,}530\ Hz \tag{7}$$

$$f_{Nyquist} = \frac{f_s}{2} = 163{,}265\ Hz \tag{8}$$

The MATLAB interface began to degrade. Beyond 163 kHz, the MATLAB plot exhibited severe aliasing, rendering a false low-frequency signal. Throughout this entire sweep, the benchtop digital storage oscilloscope maintained a perfect, uncorrupted sinusoidal trace.

At 164 kHz the EZSCOPE displayed aliasing, the waveform follows the beat phenomenon.

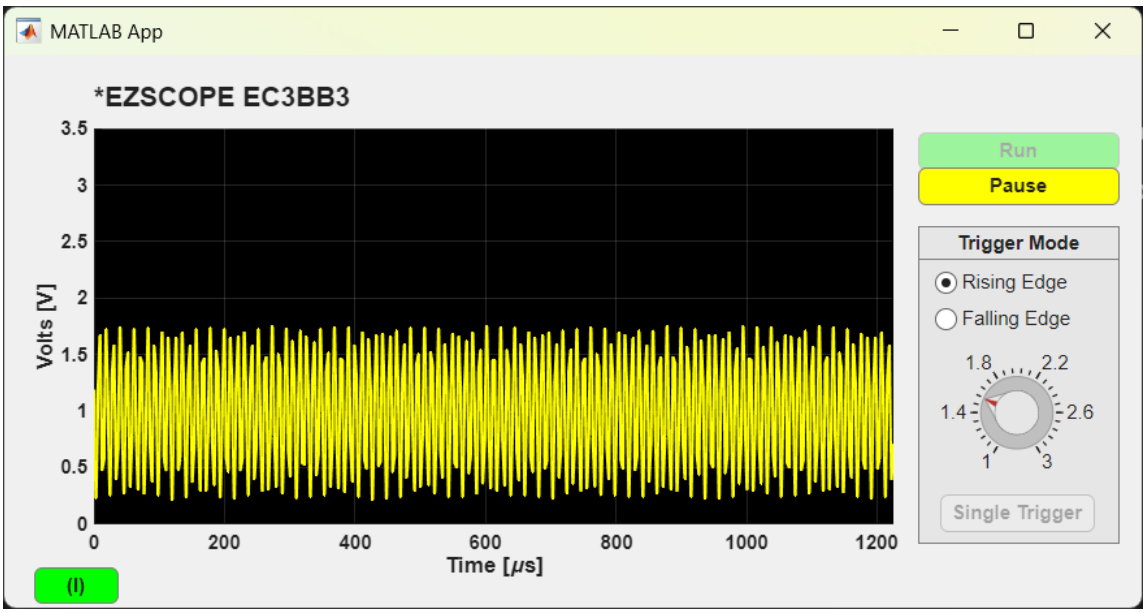

Figure 12: 163kHz sample captured on the EZSCOPE displayed aliasing, the waveform appears to follow the beat phenomenon.

164 kHz displays aliasing, the waveform follows a more apparent beat phenomenon.

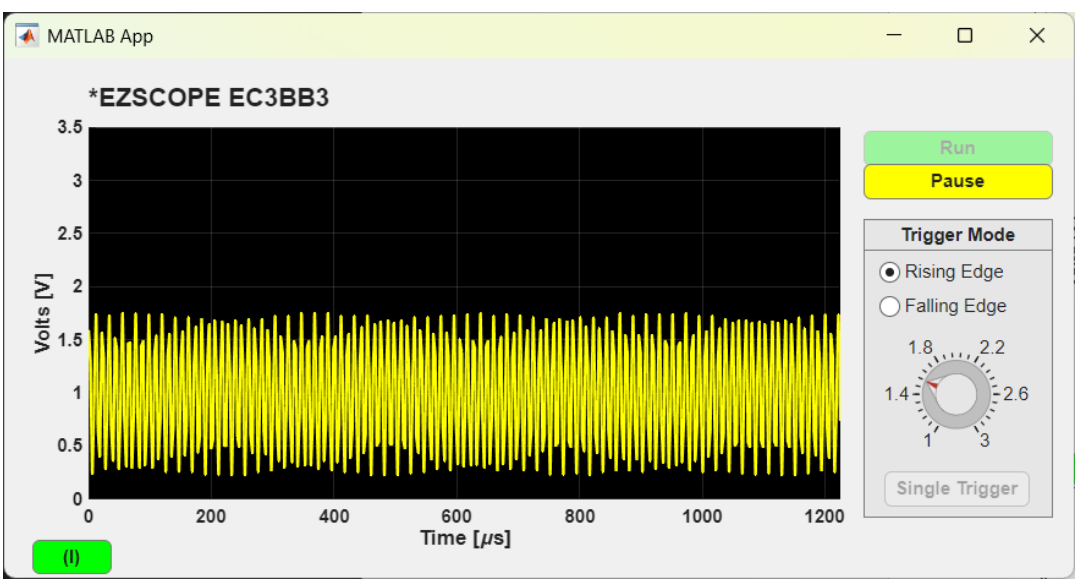

Figure 13: 164 kHz displays aliasing, the waveform follows a more apparent beat phenomenon.

A 40 kHz sample displays some folding of frequencies; this can be considered when the experimental aliasing begins.

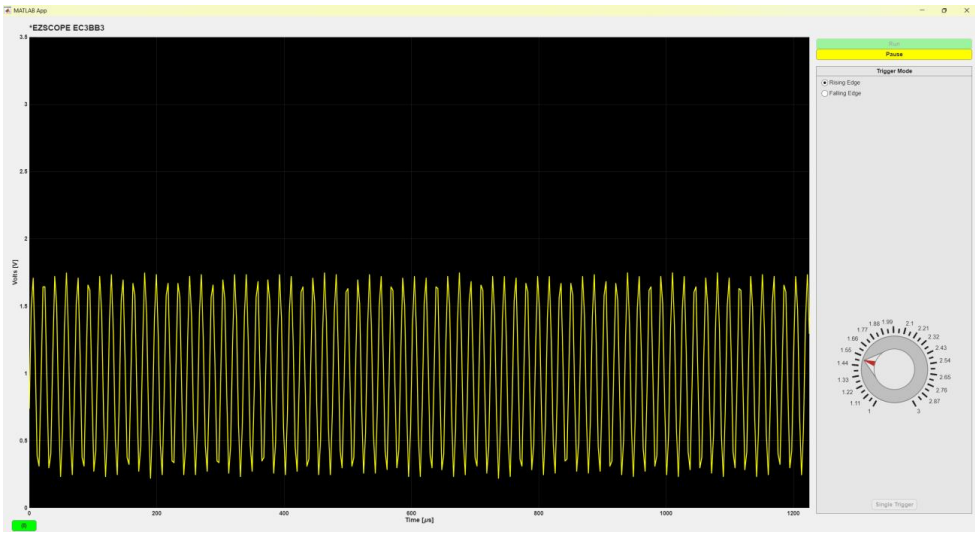

Figure 14: 40 kHz sample displays folding of frequencies and is where the experimental aliasing begins.

A 45 kHz sample displays a clear beat phenomenon the experimental aliasing continues.

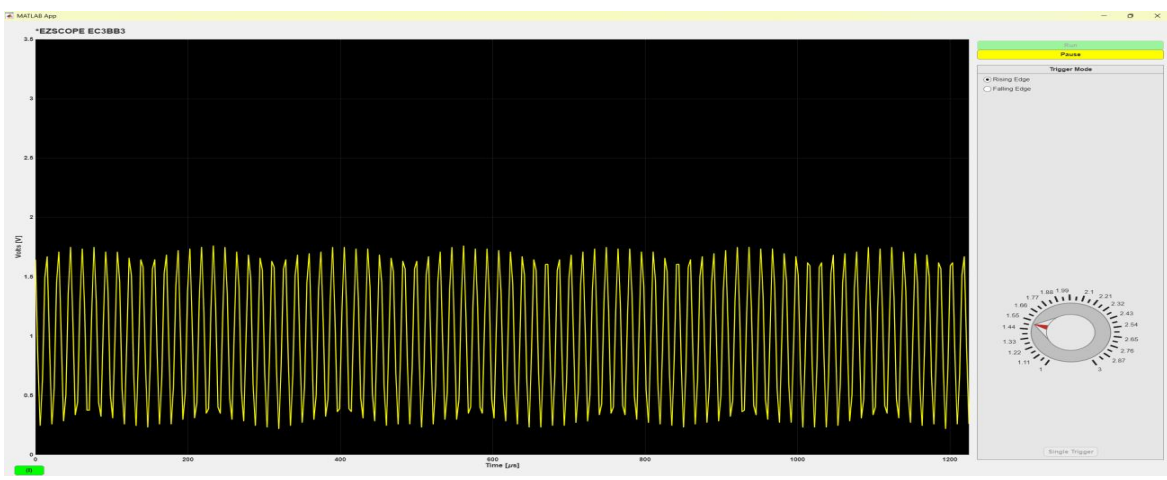

Figure 15: 45 kHz sample displays the beat phenomenon and is where the experimental aliasing happens.

The rest of the samples can be seen in Appendix D Frequency Analysis.

5 Discussion and Conclusion

The main outcome of this lab is the successful bridging of analog signal acquisition with digital software processing. The measured 49 cycles per sample significantly deviate from the theoretical 16-cycle hardware limit. This discrepancy is entirely justified by the software architecture. The 32 extra cycles are consumed by the read modify write operations required to enable the ADC, the while-loop polling the interrupt flag, shifting the 10-bit result into an 8-bit variable, and moving the data into RAM. Assuming an uncertainty of ± 1 clock cycle in the 49-cycle measurement, the calculated sample time incorporates an uncertainty of $\pm 0.06\ \mu s$, yielding a sample time of $3.06 \pm 0.06\ \mu s$.

The Timer_A module was also used to profile the UART baud rate transmission. It was determined that writing to the UCA0TXBUF register blocks the CPU for exactly 19 cycles ($\approx 1.19\ \mu s$) to load the buffer, and physical transmission inherently takes roughly 35 ms.

This measurement directly defines the operational boundaries of the oscilloscope. With a sampling rate of 326.5 ksps, the Nyquist limit dictates that any input signal exceeding 163.3 kHz will mathematically alias. Conversely, the 400-point buffer creates a maximum capture window of 1.225 ms. Therefore, signals with a frequency lower than 816 Hz may fail to display a complete period on the MATLAB UI, appearing only as partial waveforms. However, experimentally a 617.3 Hz sine wave was nearly displayed, and aliasing and folding down occurred at 40 kHz. The system cannot replace a commercial oscilloscope for transient or high-frequency continuous analysis.

Happy faces in Lab when it works after much grit, and determination :D

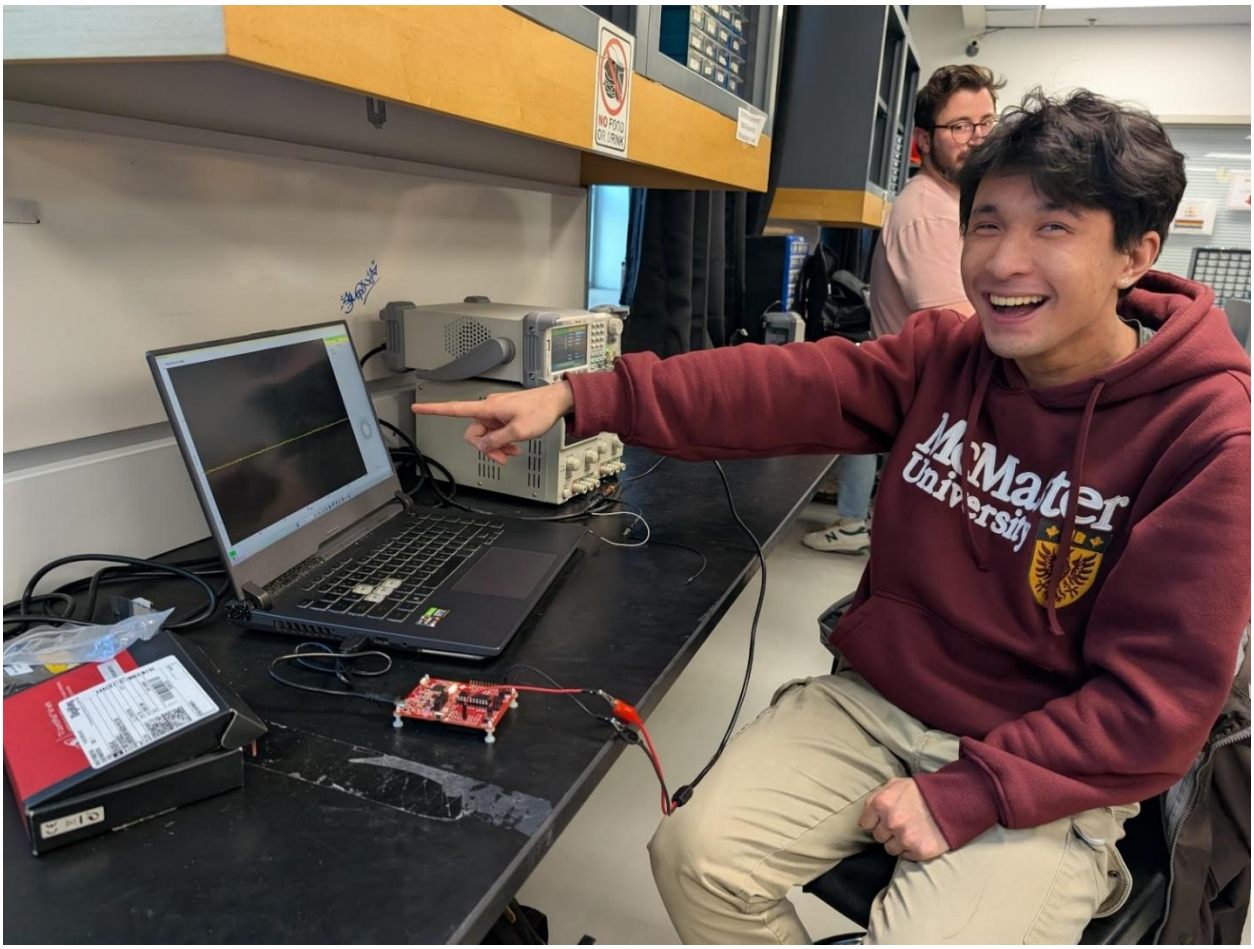

Figure 16: Silly Engineering Physics Student Smiles after returning to lab several times.

Appendices

Appendix A: Sample Calculations:

- Time Per Sample (T_{sample}):

$$T_{sample} = \frac{Cycles}{Clock\ Frequency} = \frac{49}{16{,}000{,}000\ Hz} = 3.0625\ \mu s$$

- Capture Window (T_{window}):

$$T_{window} = T_{sample} \times N_{points} = 3.0625\mu s \times 400 = 1{,}225\ \mu s = 1.225\ ms$$

- Minimum Visible Frequency (f_{min}):

$$f_{min} = \frac{1}{T_{window}} = \frac{1}{0.001225\ s} \approx 816.3\ Hz$$

- Nyquist Limit ($f_{Nyquist}$):

$$f_S = \frac{1}{T_{sample}} \approx 326{,}530\ Hz$$

$$f_{Nyquist} = \frac{f_s}{2} = 163{,}265\ Hz$$

- UART Buffer Load Time ($T_{Load\ Buffer}$):

$$T_{Load\ Buffer} = \frac{Cycles}{Clock\ Frequency} = \frac{19}{16{,}000{,}000\ Hz} = 1.1875\ \mu s$$

- UART Transmission Time:

$$Transmission\ Time = \frac{4000\ bits}{115200\ \frac{bits}{seconds}} = 0.0347\ seconds$$

Appendix B: Error Propagation Sample Calculation

Assuming an uncertainty of $\pm$ 1 clock cycle in the 49-cycle measurement ($C = 49 \pm 1$):

$$\Delta T_{sample} = \left(\frac{\Delta C}{f_{clock}}\right) = \frac{1}{16{,}000{,}000} = 0.0625\ \mu s$$

Therefore, $T_{sample} = 3.06 \pm 0.06\ \mu s$

Appendix C: System Codebase

Firmware C-Code (MSP430)

```
#include "intrinsics.h"
#include "msp430g2553.h"
#include <msp430.h>

#define NPOINTS 400

void Init(void);
void Init_ADC(void);
void Init_UART(void);

// Global Variables
volatile unsigned char v[NPOINTS];
volatile char received_char = 0;
volatile int task_ready = 0;

void main(void) {
  Init();
  Init_UART();
  Init_ADC();

  // Enable USCI_A0 RX interrupt to wait for MATLAB command
```

```
  IE2 |= UCA0RXIE;

  // Activate Global Interrupts
  __bis_SR_register(GIE);

  while (1) {
    if (task_ready == 1) {
      // Start ADC conversion sequence
      // Enable ADC interrupts to handle data collection
      ADC10CTL0 |= ADC10IE;

      // Reset task flag to wait for next command
      task_ready = 0;

    } else {
      // Enter Low Power Mode 0 with interrupts enabled while waiting
      __bis_SR_register(LPM0_bits + GIE);
    }
  }
}

void Init(void) {
  // Stop watchdog timer
  WDTCTL = WDTPW + WDTHOLD;

  // Set system clock to 16 MHz
  if (CALBC1_16MHZ==0xFF) while(1);
  DCOCTL = 0;
  BCSCTL1 = CALBC1_16MHZ;
  DCOCTL = CALDCO_16MHZ;

  // Set P1.0 to output (Green LED)
  P1DIR |= BIT0;
  P1OUT = 0x00;
}

void Init_ADC(void) {
  // ADC10CTL1: Input Channel 4 (P1.4), Mode 2 (Repeat single channel)
  ADC10CTL1 = INCH_4 + CONSEQ_2;

  // Enable analog input on P1.4
  ADC10AE0 |= BIT4;

  // ADC10CTL0: SHT_0 (4 ADC clocks), MSC (Multiple Sample Conversion), ADC
On
  ADC10CTL0 = ADC10SHT_0 + MSC + ADC10ON;

  // Enable Conversion (ENC) and Start Conversion (ADC10SC)
  ADC10CTL0 |= ENC + ADC10SC;
}

void Init_UART(void) {
  // Configure P1.1 (RX) and P1.2 (TX) for UART
  P1SEL |= BIT1 + BIT2;
  P1SEL2 |= BIT1 + BIT2;

  // Baud Rate Calculation for 16MHz / 115200
```

```
  // 16000000 / 115200 = 138.88
  UCA0BR0 = 138;      // Low byte
  UCA0BR1 = 0;        // High byte
  UCA0MCTL = UCBRS_7; // Modulation UCBRSx = 7

  // Use SMCLK for UART
  UCA0CTL1 |= UCSSEL_2;

  // Release UART Reset
  UCA0CTL1 &= ~UCSWRST;
}

// ISR: Receive Character from MATLAB
#pragma vector=USCIAB0RX_VECTOR
__interrupt void USCI0RX_ISR(void) {
  received_char = UCA0RXBUF; // Read buffer
  task_ready = 1;            // Flag main loop to start
  __bic_SR_register_on_exit(LPM0_bits); // Wake up CPU
}

// ISR: ADC Conversion Complete
#pragma vector=ADC10_VECTOR
__interrupt void ADC10_ISR(void) {
  P1OUT |= BIT0; // LED ON (Profiling start)

  unsigned int i;
  for (i = 0; i < NPOINTS; i++) {
    // Wait for conversion to complete
    while (!(ADC10CTL0 & ADC10IFG));

    // Shift 10-bit result (0-1023) to 8-bit (0-255)
    v[i] = (ADC10MEM >> 2);

    // Clear flag
    ADC10CTL0 &= ~ADC10IFG;
  }

  // Disable ADC interrupt to stop sampling
  ADC10CTL0 &= ~ADC10IE;

  // Enable UART TX interrupt to send data
  IE2 |= UCA0TXIE;
}

// ISR: Transmit Data to MATLAB
#pragma vector=USCIAB0TX_VECTOR
__interrupt void USCI0TX_ISR(void) {
  P1OUT &= ~BIT0; // LED OFF (Profiling end)

  unsigned int i;
  for (i = 0; i < NPOINTS; i++) {
    // Wait for TX buffer to be ready
    while (!(IFG2 & UCA0TXIFG));

    // Send data
    UCA0TXBUF = v[i];
  }
```

```
  // Disable TX interrupt after sending all points
  IE2 &= ~UCA0TXIE;
}
```

MATLAB Application Designer Code

```
classdef Copy_6_of_EZScope < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                 matlab.ui.Figure
        PauseButton              matlab.ui.control.Button
        TriggerModeButtonGroup   matlab.ui.container.ButtonGroup
        Knob2                    matlab.ui.control.Knob
        SingleTriggerButton      matlab.ui.control.Button
        FallingEdgeButton        matlab.ui.control.RadioButton
        RisingEdgeButton         matlab.ui.control.RadioButton
        IButton                  matlab.ui.control.Button
        RunButton                matlab.ui.control.Button
        UIAxes                   matlab.ui.control.UIAxes
    end


    properties (Access = private)
        comport % Object for serial communication
        clockfreq = 16e6 % Clock frequency
        npoints = 400; % Number of data points matching C code
        % UPDATED: Set to 49 based on your latest measurement
        daccycles = 49;
        stop_req = false; %Flag to raise to stop continuous loop
    end

% Callbacks that handle component events
    methods (Access = private)
        % Code that executes after component creation
        function startupFcn(app)
            % Configure Serial Port (Adjust COM port as needed)
            % Check Device Manager for the correct port (e.g., COM3, COM4)
            try
                % CHANGE "COM3" TO YOUR ACTUAL PORT (e.g., "COM4")
                app.comport = serialport("COM4", 115200);
                configureTerminator(app.comport, "LF");n

                % Set initial button states
                app.PauseButton.Enable = 'off';
            catch
                uialert(app.UIFigure, "Could not connect to COM port. Check
Device Manager.", "Connection Error");
            end
        end
    end



    % Callbacks that handle component events
    methods (Access = private)

        % Button pushed function: RunButton
        function RunButtonPushed(app, event)
            % 1. CONNECT ONLY IF NOT ALREADY CONNECTED
```

```
            % Check if app.comport is empty or if the connection is invalid
(closed)
            if isempty(app.comport) || ~isvalid(app.comport)
                try
                    app.comport = serialport("COM4", 115200);
                    configureTerminator(app.comport, "LF");
                catch
                    uialert(app.UIFigure, "Could not open COM4. Check
connection.", "Error");
                    return;
                end
            end

            % 2. Update UI States
            app.RunButton.Enable = 'off';
            app.SingleTriggerButton.Enable = 'off';
            app.PauseButton.Enable = 'on';
            app.IButton.Enable = 'on';

            % 3. Reset Stop Flag
            app.stop_req = false;

            % 4. Time Vector Calculation (Microseconds)
            % Time per sample = Cycles / Frequency
            dt_seconds = app.daccycles / app.clockfreq;
            dt_micros = dt_seconds * 1e6; % Convert to microseconds

            % Total time in microseconds
            total_time_us = app.npoints * dt_micros;

            % Create time vector (0 to Total Time)
            full_time_vec = linspace(0, total_time_us, app.npoints);

            while ~app.stop_req
                % 2. Send Trigger Character to MSP430
                try
                    write(app.comport, 'U', "char");

                    % 3. Read Data (Blocking read of 400 bytes)
                    raw_data = read(app.comport, app.npoints, "uint8");

                    % 4. Convert to Voltage
                    % MSP430 C code shifts right by 2, so 0-255 maps to 0-
3.3V roughly
                    % V = (Data / 255) * 3.3V (Approx Vcc)
                    voltage_data = (double(raw_data) ./ 255) * 3.3;

                    % 5. PLOT vs TIME
                    % Use the time vector as the X-axis
                    plot(app.UIAxes, full_time_vec, voltage_data, '-y',
'LineWidth', 1.5);

                    % Update Axis Limits (X is now in Microseconds)
                    ylim(app.UIAxes, [0 3.5]);
                    xlim(app.UIAxes, [0 total_time_us]); % Lock to 0 -
1225us
                    grid(app.UIAxes, 'on');
```

```
                    % Force MATLAB to update the UI and check for clicks
                        drawnow;

                catch
                    app.stop_req = true; % Stop if serial error occurs
                end
            end



            % 6. Reset UI States when loop finishes
            app.RunButton.Enable = 'on';
            app.SingleTriggerButton.Enable = 'on';
            app.PauseButton.Enable = 'off';
            app.IButton.Enable = 'on';
        end

        % Button pushed function: IButton
        function IButtonPushed(app, event)
        app.stop_req = true; % Ensure loop stops
            pause(0.1); % Give loop time to exit
            if ~isempty(app.comport) && isvalid(app.comport)
                delete(app.comport);
            end
            delete(app.UIFigure);
        end

        % Button pushed function: SingleTriggerButton
        function SingleTriggerButtonPushed(app, event)
            % 1. CONNECT ONLY IF NOT ALREADY CONNECTED
            % Check if app.comport is empty or if the connection is invalid
(closed)
            if isempty(app.comport) || ~isvalid(app.comport)
                try
                    app.comport = serialport("COM4", 115200);
                    configureTerminator(app.comport, "LF");
                catch
                    uialert(app.UIFigure, "Could not open COM4. Check
connection.", "Error");
                    return;
                end
            end

            % 2. Send Trigger Character to MSP430
            try
                write(app.comport, 'U', "char");

                % 3. Read Data (Blocking read of 400 bytes)
                raw_data = read(app.comport, app.npoints, "uint8");
            catch
                uialert(app.UIFigure, "Failed to read data from MSP430.",
"Read Error");
                return;
            end
            % 4. Convert to Voltage
            % MSP430 C code shifts right by 2, so 0-255 maps to 0-3.3V
roughly
```

```
            % V = (Data / 255) * 3.3V (Approx Vcc)
            voltage_data = (double(raw_data) ./ 255) * 3.3;

            % 5. Software Triggering logic
            % Get selected mode from the Button Group
            trigger_mode = app.TriggerModeButtonGroup.SelectedObject.Text;
            % Read from the Edit Field
            trigger_level = app.Knob2.Value;
            trigger_index = 1;

            % Search for the trigger point
            for i = 1:(app.npoints - 1)
                val_index = voltage_data(i);
                val_next = voltage_data(i+1);

                if strcmp(trigger_mode, 'Rising Edge')
                    % Rising: Current < Level AND Next >= Level
                    if (val_index < trigger_level) && (val_next >=
trigger_level)
                        trigger_index = i;
                        break;
                    end
                elseif strcmp(trigger_mode, 'Falling Edge')
                    % Falling: Current > Level AND Next <= Level
                    if (val_index > trigger_level) && (val_next <=
trigger_level)
                        trigger_index = i;
                        break;
                    end
                end
            end

            % 5. Slice Data
            sliced_data = voltage_data(trigger_index:end);

            % 6. Create Sliced Time Vector (Microseconds)
            % Time step in microseconds
            dt_us = (app.daccycles / app.clockfreq) * 1e6;

            % Create vector from 0 to length of sliced data
            sliced_time = (0:(length(sliced_data)-1)) * dt_us;

            % 7. Plot
            plot(app.UIAxes, sliced_time, sliced_data, '-y', 'LineWidth',
1.5);

            % Limits
            ylim(app.UIAxes, [0 3.5]);
            % Set X limit to the full window size so the scale doesn't jump
            total_window_us = app.npoints * dt_us;
            xlim(app.UIAxes, [0 total_window_us]);
            grid(app.UIAxes, 'on');

            % Draw the trigger level line for visual reference
            yline(app.UIAxes, trigger_level, 'w', 'Trigger Level');
        end
```

```
        % Button pushed function: PauseButton
        function PauseButtonPushed(app, event)
            app.stop_req = true;
        end
    end

    % Component initialization
    methods (Access = private)

        % Create UIFigure and components
        function createComponents(app)

            % Create UIFigure and hide until all components are created
            app.UIFigure = uifigure('Visible', 'off');
            app.UIFigure.Color = [0.9412 0.9412 0.9412];
            app.UIFigure.Position = [100 100 700 339];
            app.UIFigure.Name = 'MATLAB App';

            % Create UIAxes
            app.UIAxes = uiaxes(app.UIFigure);
            title(app.UIAxes, '*EZSCOPE EC3BB3')
            xlabel(app.UIAxes, 'Time [\mus]')
            ylabel(app.UIAxes, 'Volts [V]')
            zlabel(app.UIAxes, 'Z')
            app.UIAxes.AmbientLightColor = [0.502 0.502 0.502];
            app.UIAxes.FontWeight = 'bold';
            app.UIAxes.Color = [0 0 0];
            app.UIAxes.TitleHorizontalAlignment = 'left';
            app.UIAxes.GridColor = [0.9412 0.9412 0.9412];
            app.UIAxes.XGrid = 'on';
            app.UIAxes.YGrid = 'on';
            app.UIAxes.TitleFontSizeMultiplier = 1.4;
            app.UIAxes.Position = [19 19 534 306];

            % Create RunButton
            app.RunButton = uibutton(app.UIFigure, 'push');
            app.RunButton.ButtonPushedFcn = createCallbackFcn(app,
@RunButtonPushed, true);
            app.RunButton.BackgroundColor = [0 1 0];
            app.RunButton.FontWeight = 'bold';
            app.RunButton.Position = [563 271 124 23];
            app.RunButton.Text = 'Run';

            % Create IButton
            app.IButton = uibutton(app.UIFigure, 'push');
            app.IButton.ButtonPushedFcn = createCallbackFcn(app,
@IButtonPushed, true);
            app.IButton.BackgroundColor = [0 1 0];
            app.IButton.FontWeight = 'bold';
            app.IButton.Position = [19 4 52 23];
            app.IButton.Text = '(I)';

            % Create TriggerModeButtonGroup
            app.TriggerModeButtonGroup = uibuttongroup(app.UIFigure);
            app.TriggerModeButtonGroup.TitlePosition = 'centertop';
            app.TriggerModeButtonGroup.Title = 'Trigger Mode';
```

```
            app.TriggerModeButtonGroup.BackgroundColor = [0.902 0.902
0.902];
            app.TriggerModeButtonGroup.FontWeight = 'bold';
            app.TriggerModeButtonGroup.Position = [563 41 124 195];

            % Create RisingEdgeButton
            app.RisingEdgeButton =
uiradiobutton(app.TriggerModeButtonGroup);
            app.RisingEdgeButton.Text = 'Rising Edge';
            app.RisingEdgeButton.Position = [11 149 86 22];
            app.RisingEdgeButton.Value = true;

            % Create FallingEdgeButton
            app.FallingEdgeButton =
uiradiobutton(app.TriggerModeButtonGroup);
            app.FallingEdgeButton.Text = 'Falling Edge';
            app.FallingEdgeButton.Position = [11 127 88 22];

            % Create SingleTriggerButton
            app.SingleTriggerButton = uibutton(app.TriggerModeButtonGroup,
'push');
            app.SingleTriggerButton.ButtonPushedFcn = createCallbackFcn(app,
@SingleTriggerButtonPushed, true);
            app.SingleTriggerButton.FontWeight = 'bold';
            app.SingleTriggerButton.Position = [14 7 95 23];
            app.SingleTriggerButton.Text = 'Single Trigger';

            % Create Knob2
            app.Knob2 = uiknob(app.TriggerModeButtonGroup, 'continuous');
            app.Knob2.Limits = [1 3];
            app.Knob2.Position = [39 58 45 45];
            app.Knob2.Value = 1.5;

            % Create PauseButton
            app.PauseButton = uibutton(app.UIFigure, 'push');
            app.PauseButton.ButtonPushedFcn = createCallbackFcn(app,
@PauseButtonPushed, true);
            app.PauseButton.BackgroundColor = [1 1 0];
            app.PauseButton.FontWeight = 'bold';
            app.PauseButton.Position = [563 249 124 23];
            app.PauseButton.Text = 'Pause';

            % Show the figure after all components are created
            app.UIFigure.Visible = 'on';
        end
    end

    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = Copy_6_of_EZScope

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
```

```
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```

Cycle Profiling

```
#include "intrinsics.h"
#include "msp430g2553.h"
#include <msp430.h>

// Note: stdio.h is removed because println/printf is too heavy
// for basic timing and usually requires complex UART setup.

#define NPOINTS 400

void Init_System(void);

// Global Variables for Profiling (View these in CCS "Expressions" Window)
volatile unsigned int start_time;
volatile unsigned int end_time;
volatile unsigned int total_cycles_adc;
volatile unsigned int total_cycles_uart;
volatile int adc_result;

void main(void) {
    Init_System();

    // --- SETUP TIMER TO COUNT CYCLES ---
    // TASSEL_2 = Use SMCLK (16MHz)
    // MC_2     = Continuous Mode (Counts 0 to 65535 repeatedly)
    TA0CTL = TASSEL_2 + MC_2 + TACLR;

    while (1) {
        // ========================================================
        // MEASURE ADC (One Sample)
        // ========================================================

        // 1. Capture Timer Value BEFORE starting
        start_time = TA0R;

        // 2. Start ADC (Enable + Start Conversion)
        ADC10CTL0 |= ENC + ADC10SC;

        // 3. Wait for conversion to complete
        while (!(ADC10CTL0 & ADC10IFG));

        // 4. Read data
        adc_result = ADC10MEM >> 2;

        // 5. Clear flag
        ADC10CTL0 &= ~ADC10IFG;

        // 6. Capture Timer Value AFTER
        end_time = TA0R;

        // 7. Calculate Difference
        total_cycles_adc = end_time - start_time;

        // Stop ADC logic for the next loop
        ADC10CTL0 &= ~ENC;
```

```
        // ========================================================
        // MEASURE UART (One Byte CPU Load)
        // ========================================================

        // 1. Capture Timer Value BEFORE
        start_time = TA0R;

        // 2. Wait for TX buffer to be ready
        while (!(IFG2 & UCA0TXIFG));

        // 3. Write data (This puts the byte in the buffer)
        UCA0TXBUF = 'A';

        // 4. Capture Timer Value AFTER
        // Note: This measures how long the CPU is blocked.
        // It does NOT measure the time for the bit to travel down the wire.
        end_time = TA0R;

        // 5. Calculate Difference
        total_cycles_uart = end_time - start_time;

        // Delay to allow physical transmission to finish before next loop
        __delay_cycles(2000);

        // BREAKPOINT HERE: Inspect 'total_cycles_adc' and
'total_cycles_uart'
        __no_operation();
    }
}

void Init_System(void) {
    WDTCTL = WDTPW + WDTHOLD;
    if (CALBC1_16MHZ==0xFF) while(1);
    DCOCTL = 0;
    BCSCTL1 = CALBC1_16MHZ;
    DCOCTL = CALDCO_16MHZ;

    // ADC Setup
    // Add ADC10SSEL_2 to select SMCLK (16MHz)
    ADC10CTL1 = INCH_4 + CONSEQ_0 + ADC10SSEL_2;
    ADC10AE0 |= BIT4;
    ADC10CTL0 = ADC10SHT_0 + MSC + ADC10ON;

    // UART Setup
    P1SEL |= BIT1 + BIT2;
    P1SEL2 |= BIT1 + BIT2;
    UCA0BR0 = 138; UCA0BR1 = 0; UCA0MCTL = UCBRS_7;
    UCA0CTL1 |= UCSSEL_2;
    UCA0CTL1 &= ~UCSWRST;
}
```

Appendix D: Frequency Analysis

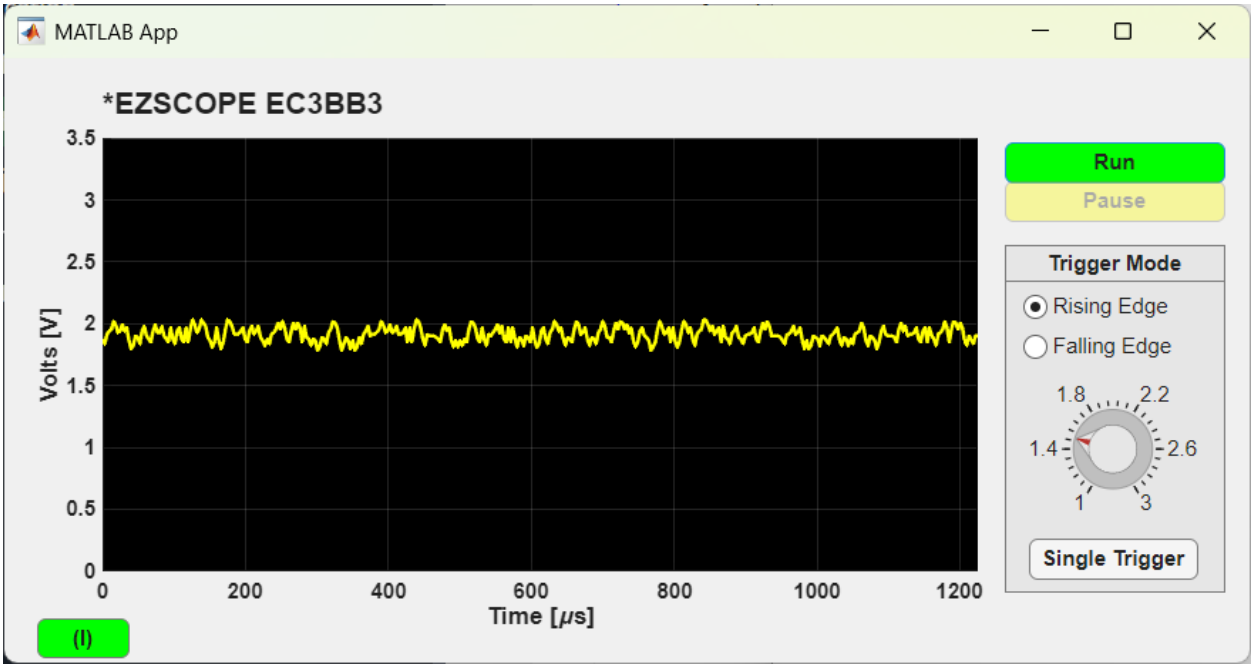

Figure 17: EZSCOPE Noise Sample

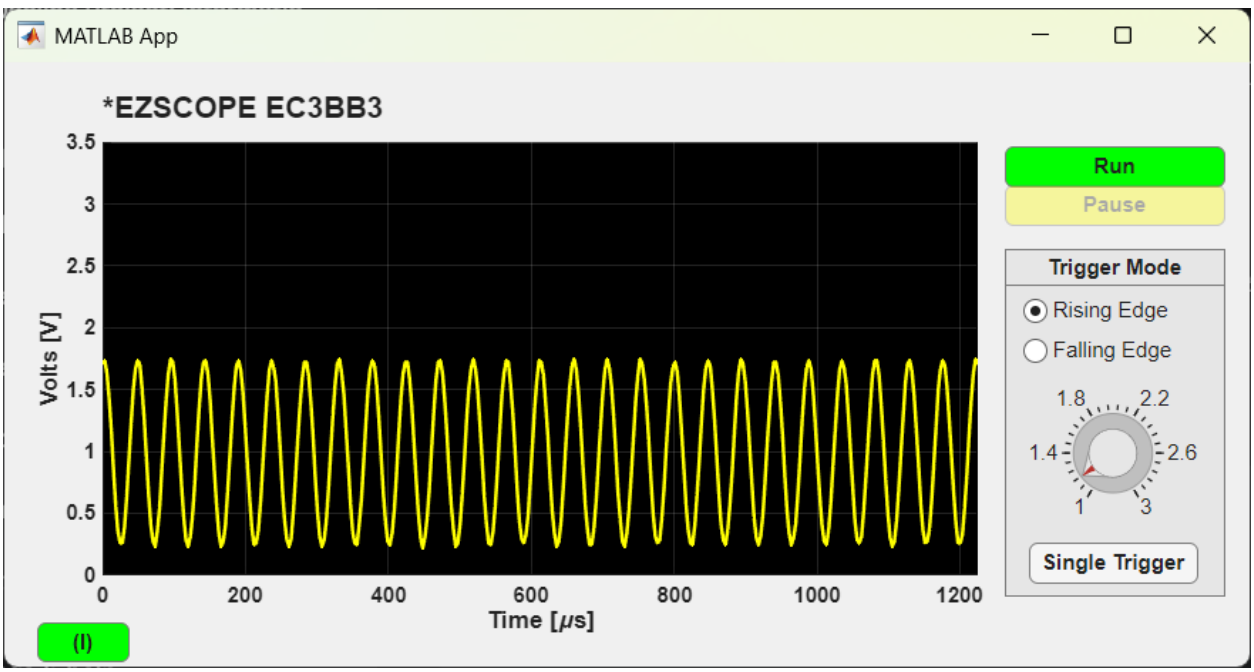

Figure 18: EZSCOPE 15kHz Sample

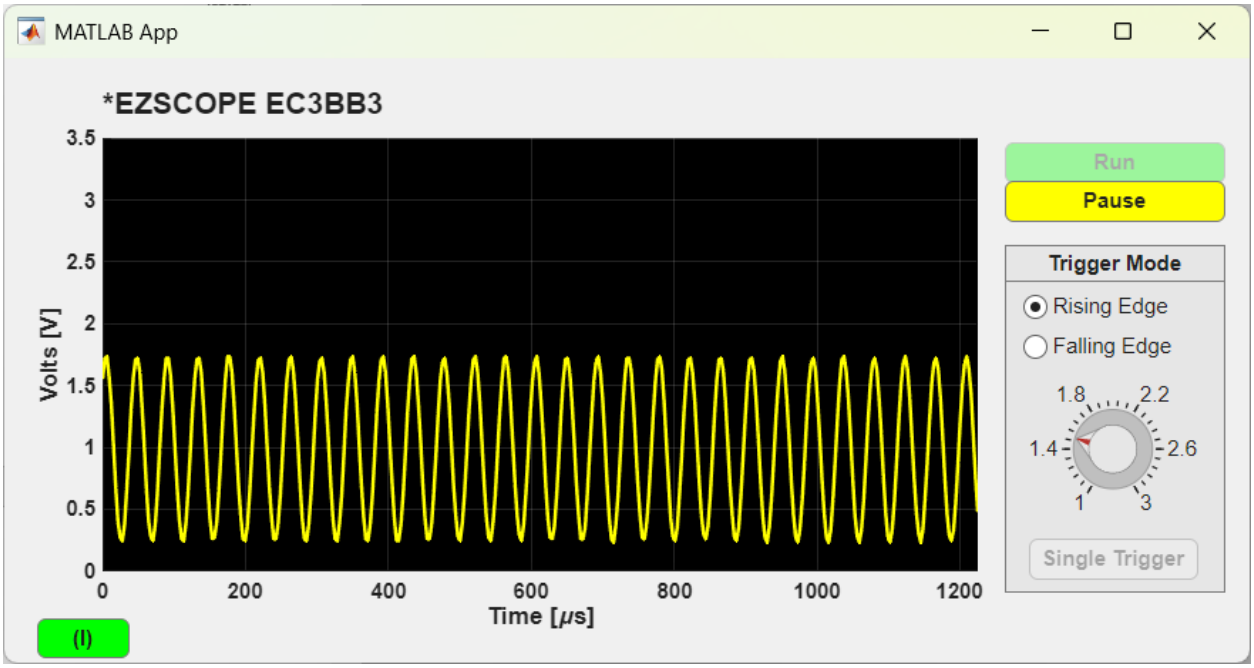

Figure 19:EZSCOPE 16.4kHz Sample

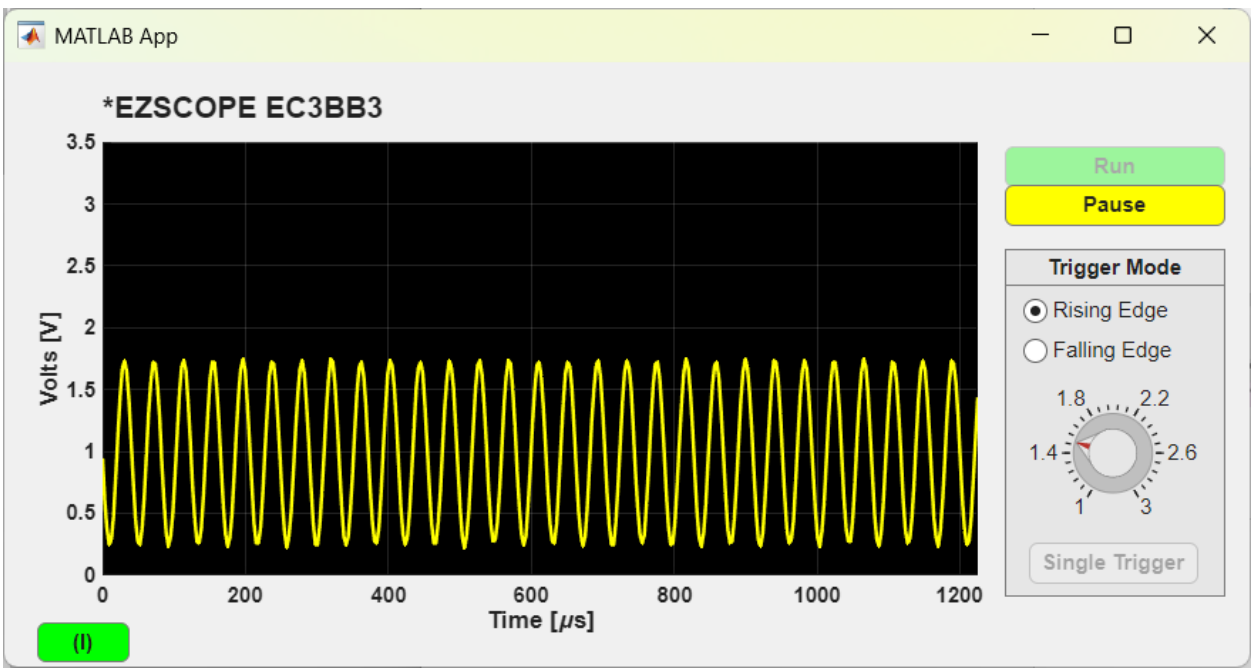

Figure 20: EZSCOPE 17 kHz Sample

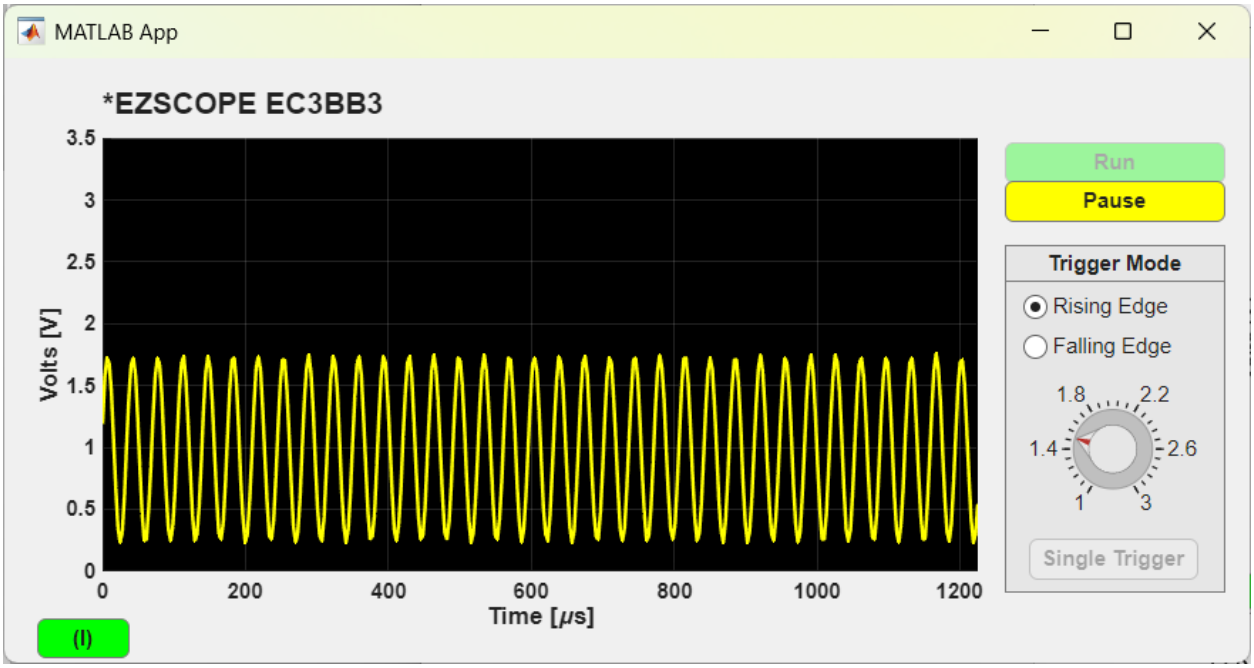

Figure 21: EZSCOPE 20 kHz Sample

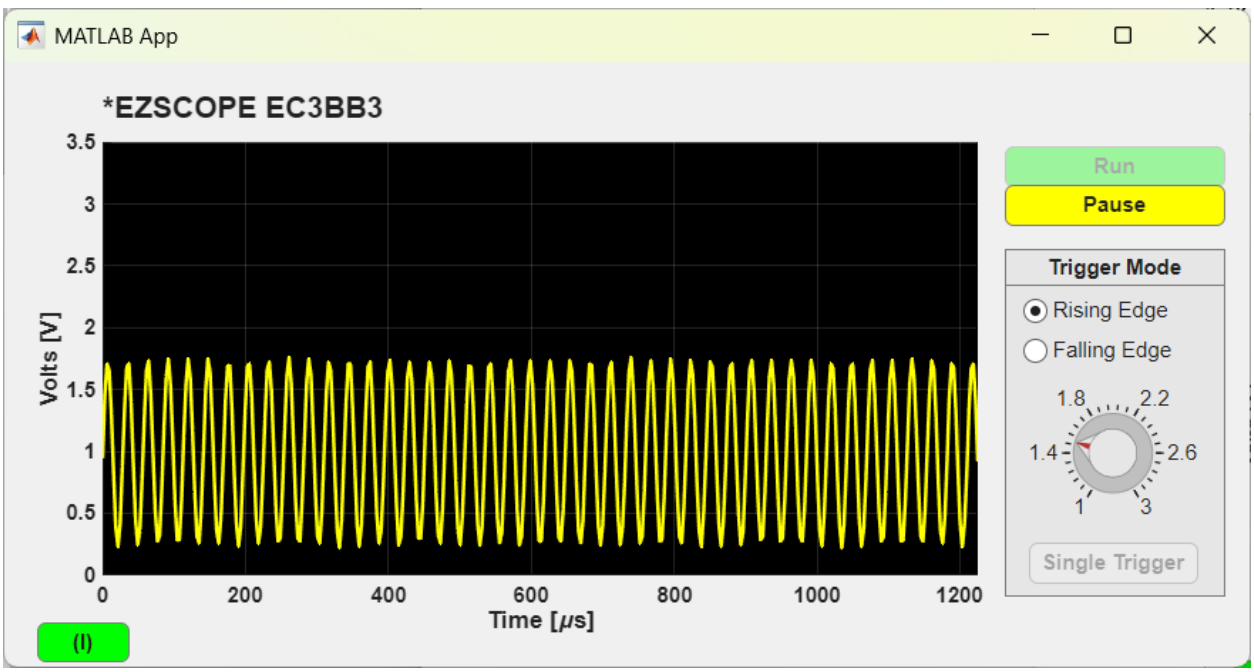

Figure 22: EZSCOPE 25 kHz Sample

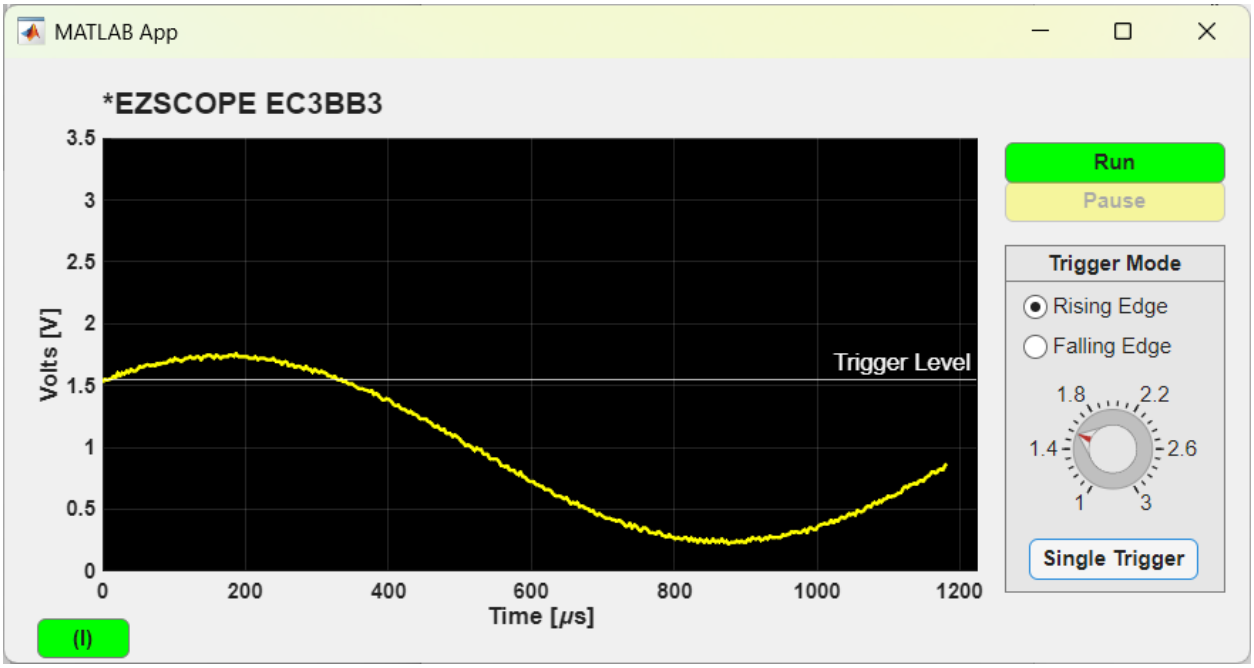

Figure 23: EZSCOPE 500 Hz Sample

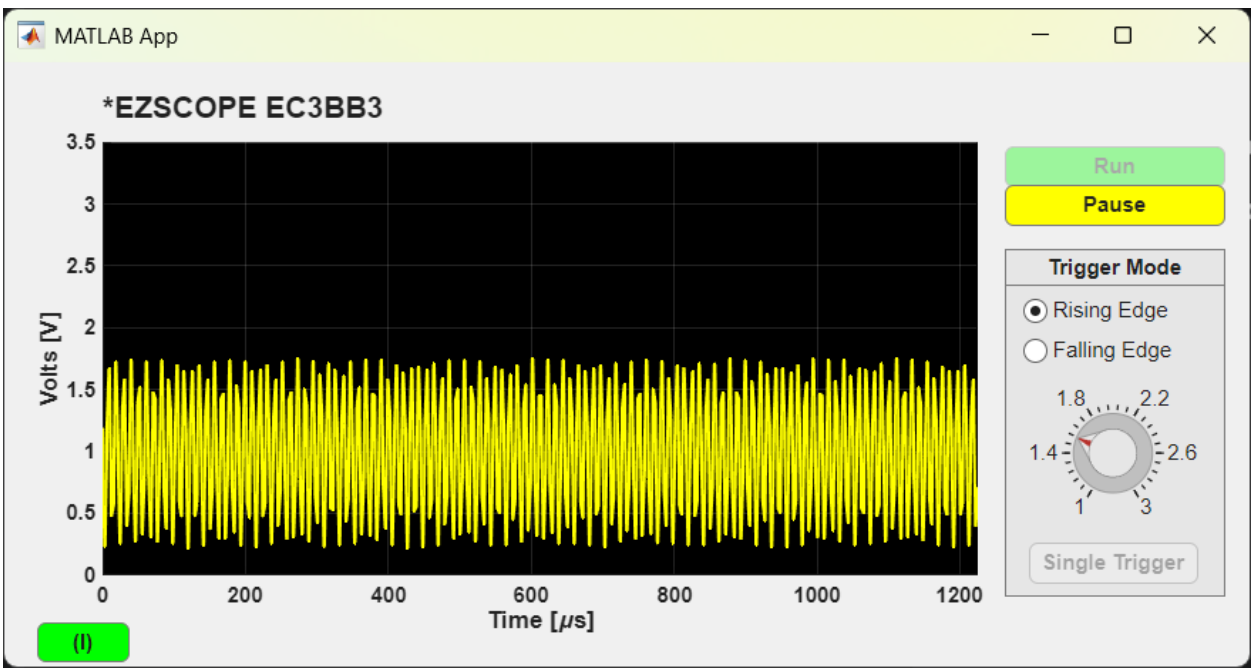

Figure 24: EZSCOPE 163 kHz Sample

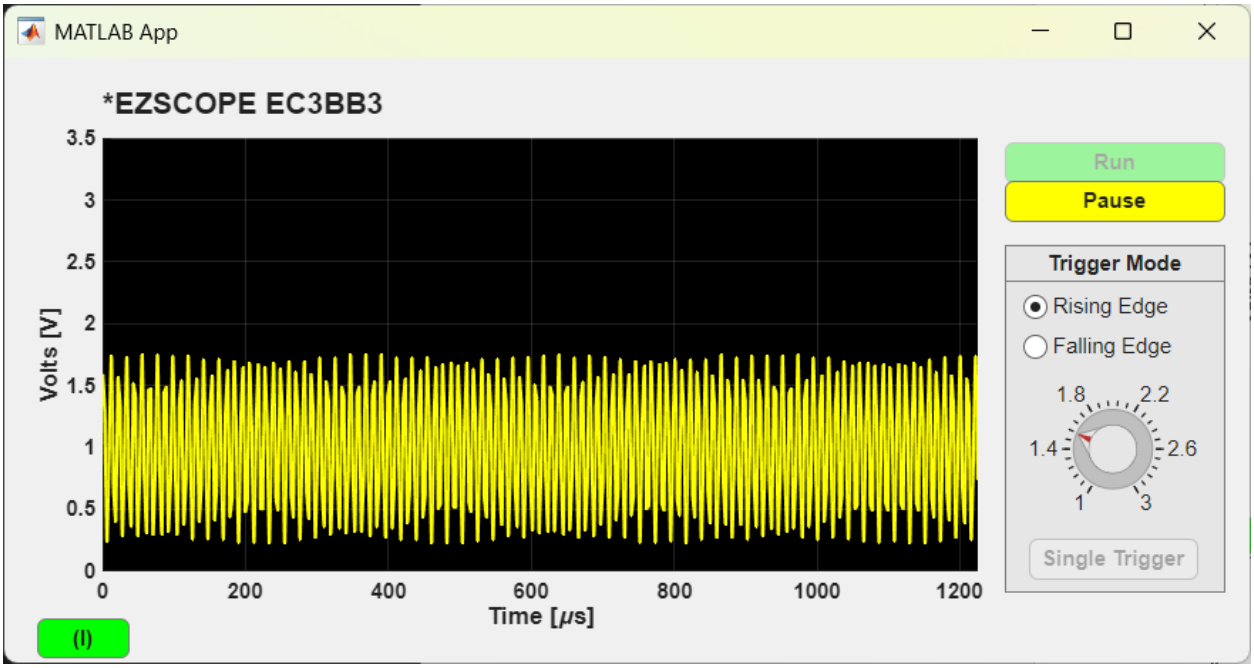

Figure 25: EZSCOPE 164 kHz Sample

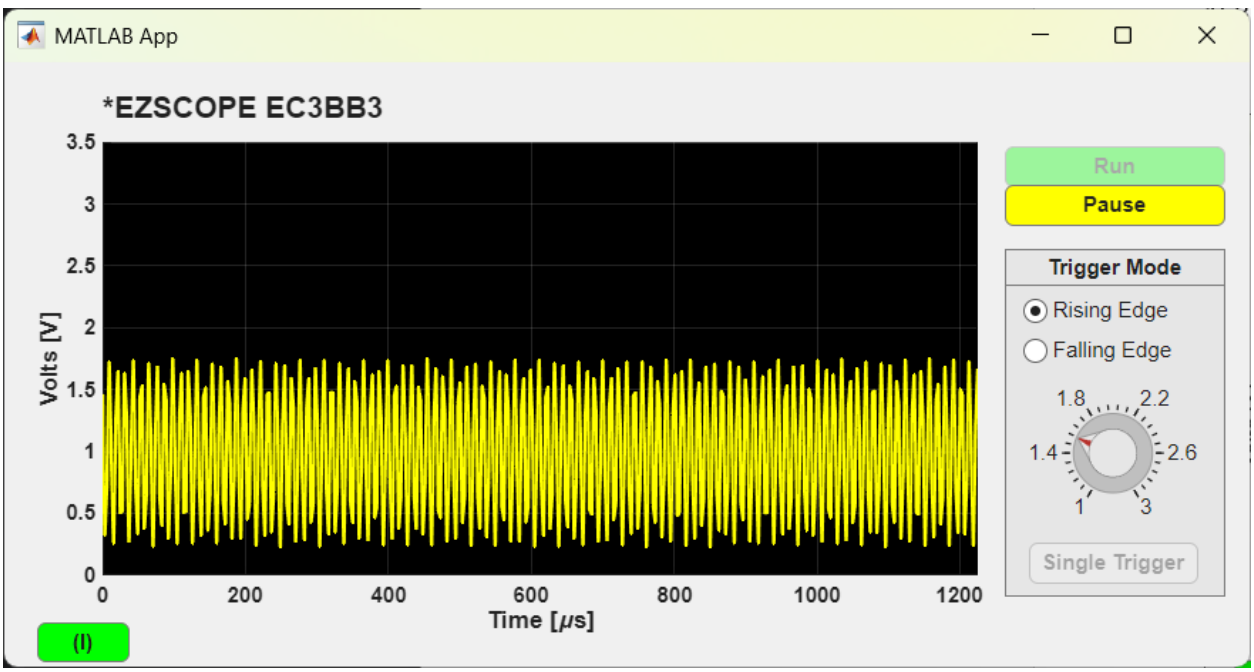

Figure 26: EZSCOPE 166 kHz Sample

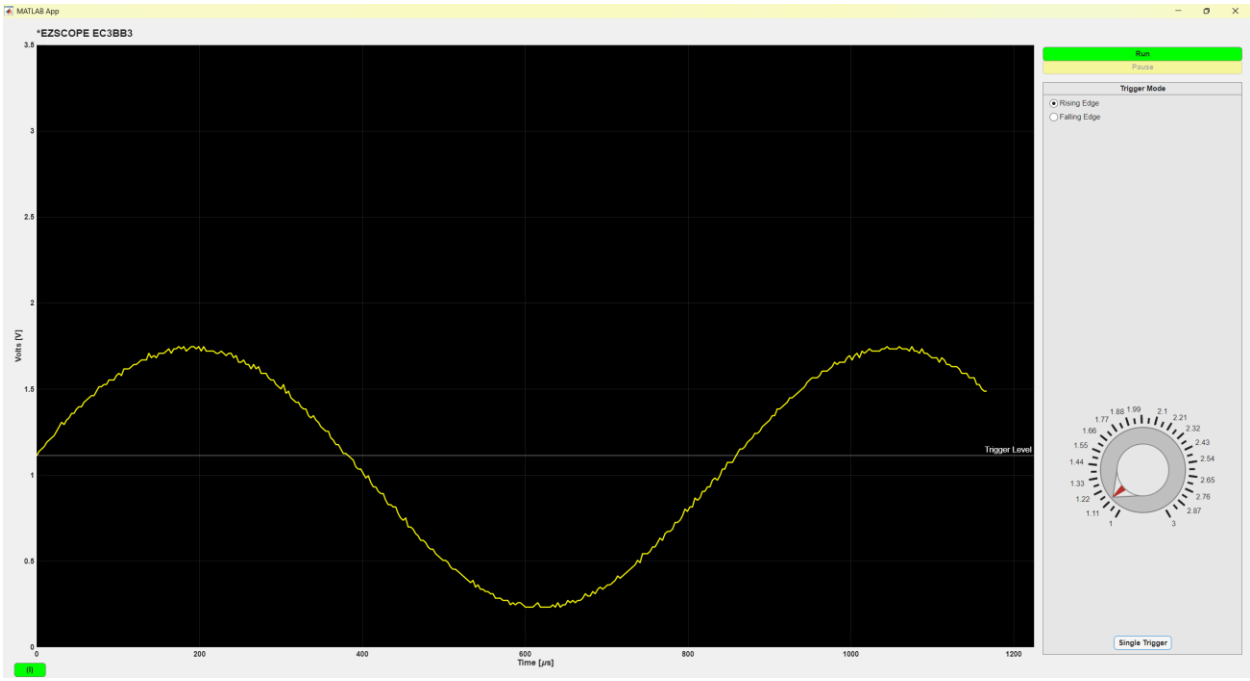

Figure 27: EZSCOPE 816 Hz Sample

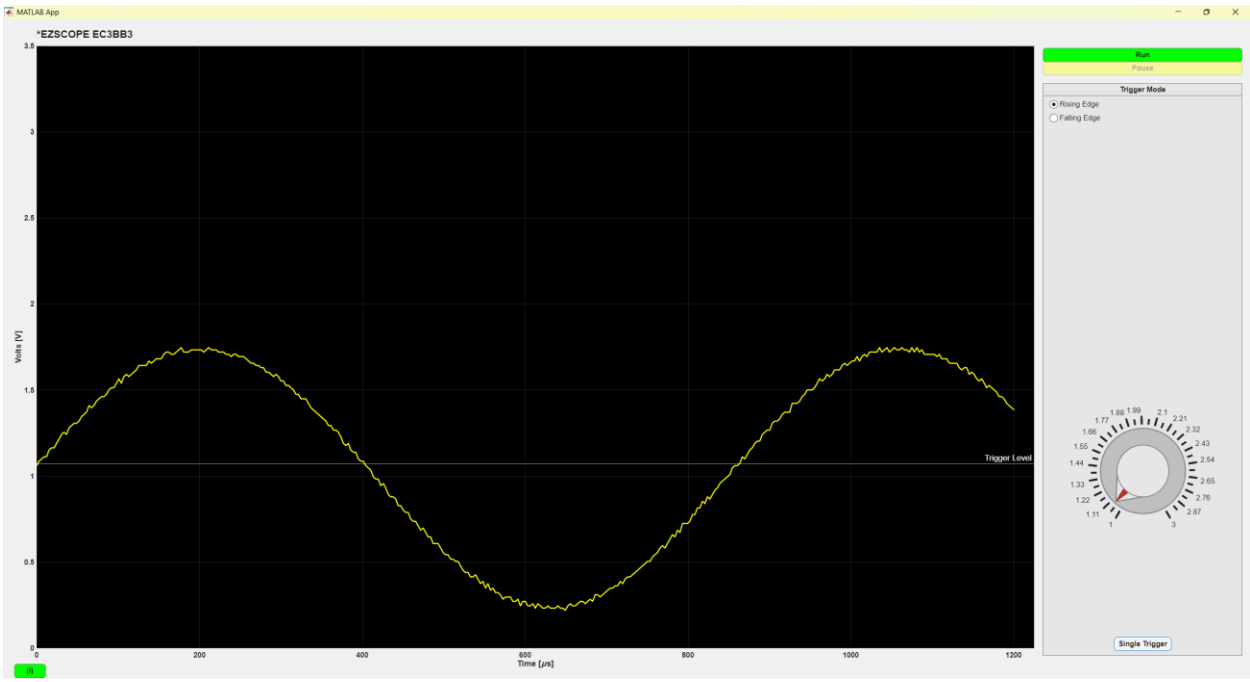

Figure 28: EZSCOPE 817 Sample

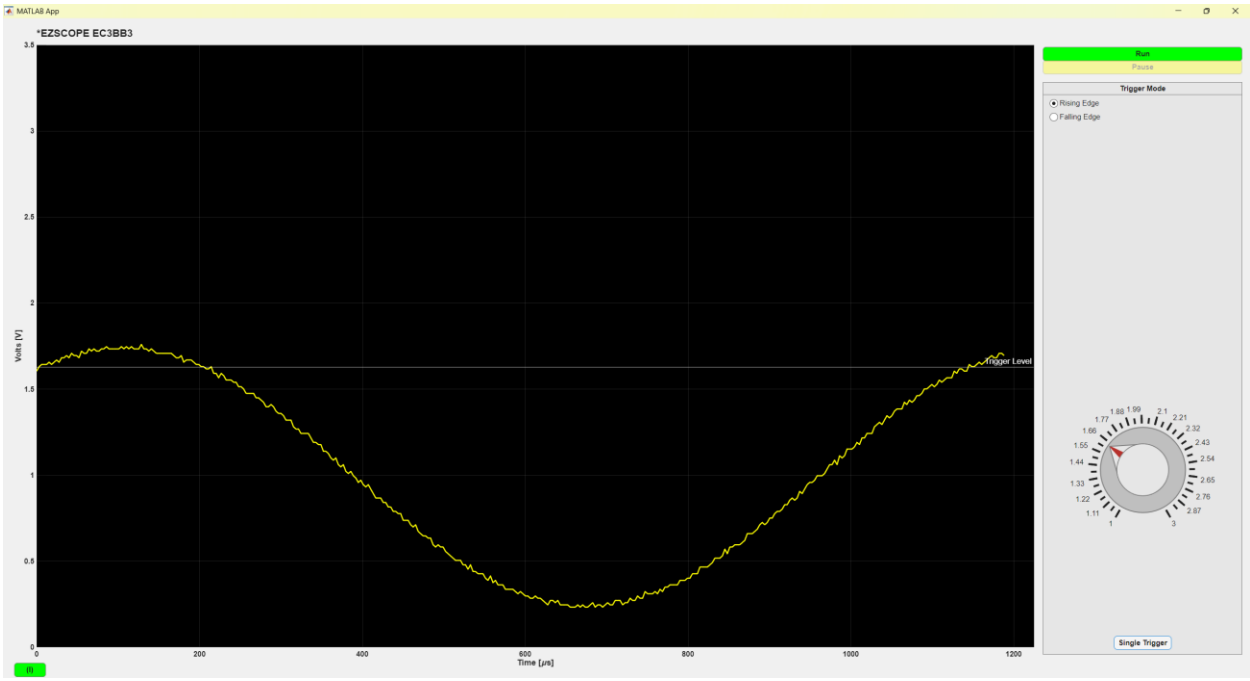

Figure 29: EZSCOPE 617.3 Hz Sample

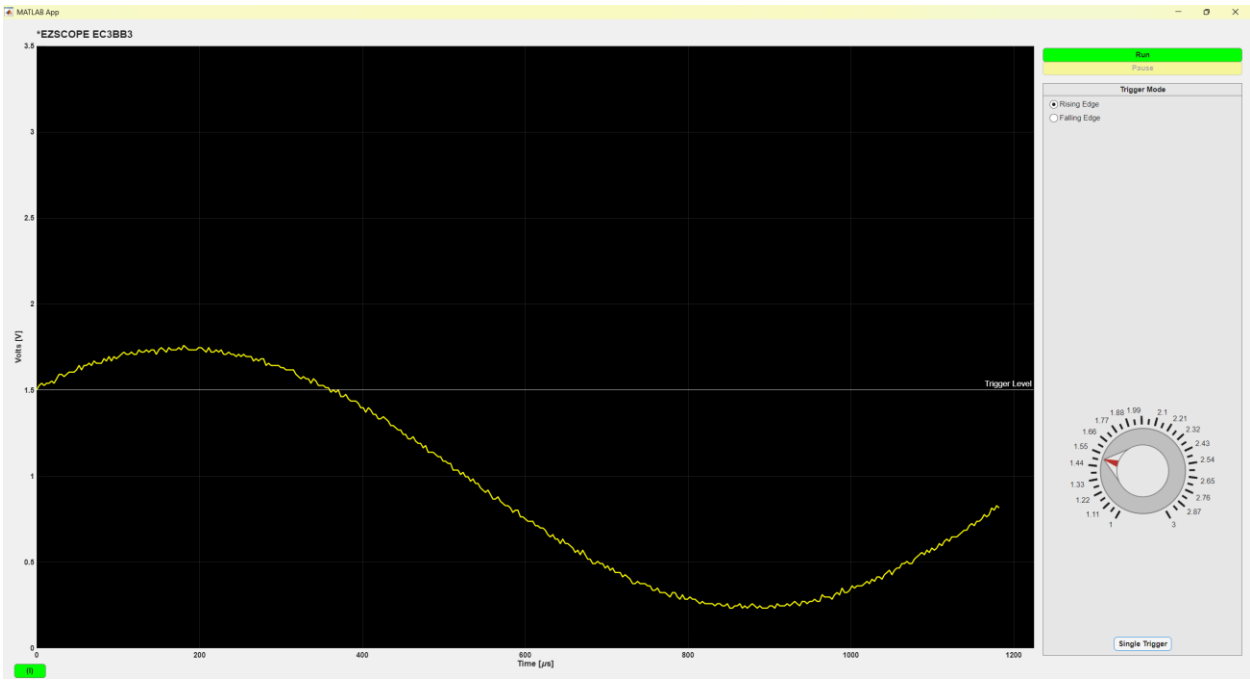

Figure 30: EZSCOPE 500 Hz Sample

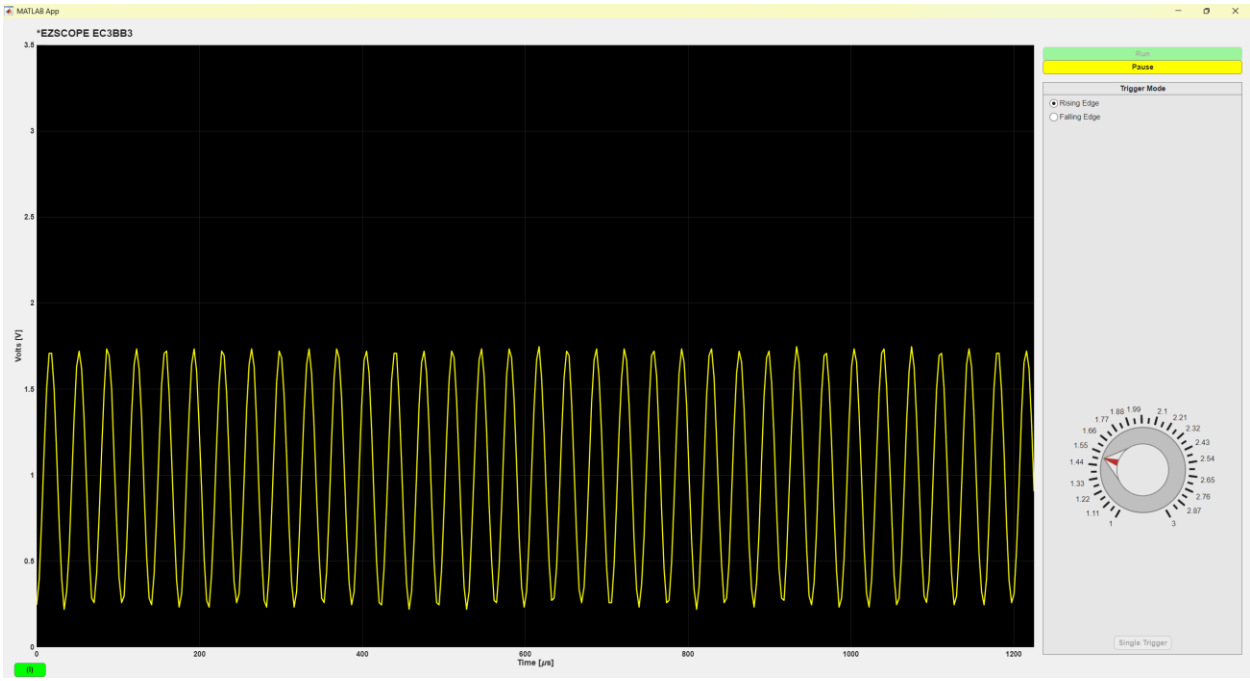

Figure 31: EZSCOPE 20 kHz Sample

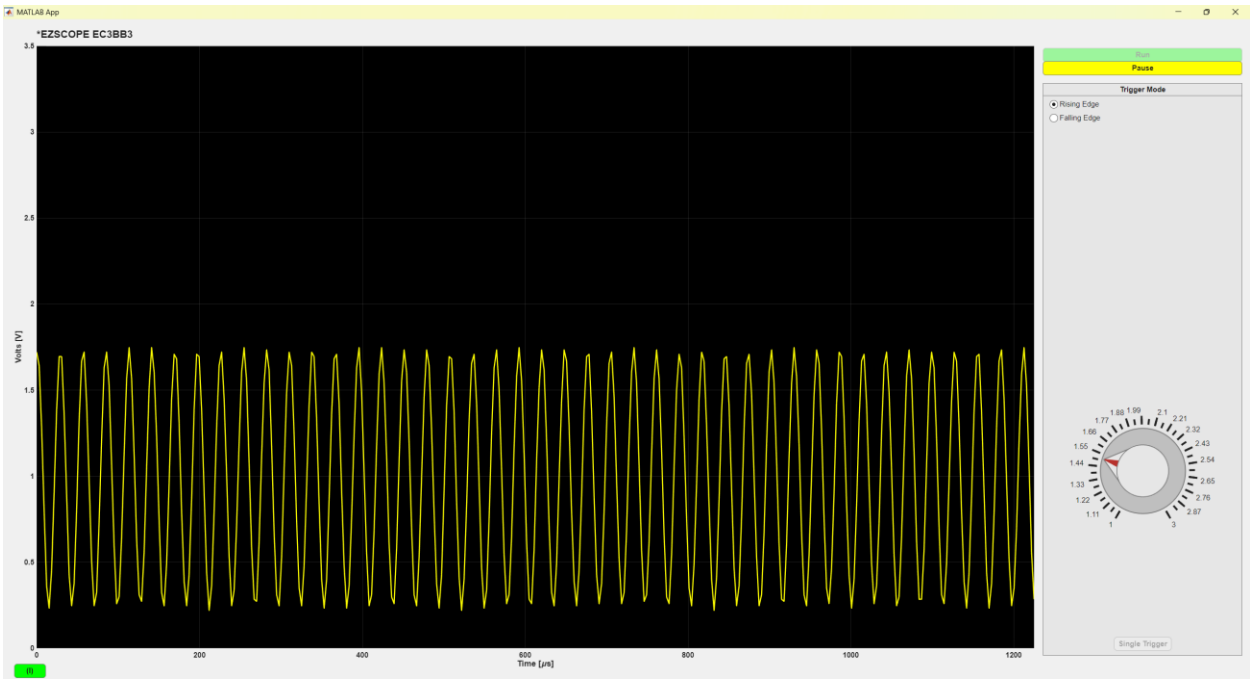

Figure 32: EZSCOPE 25 kHz Sample

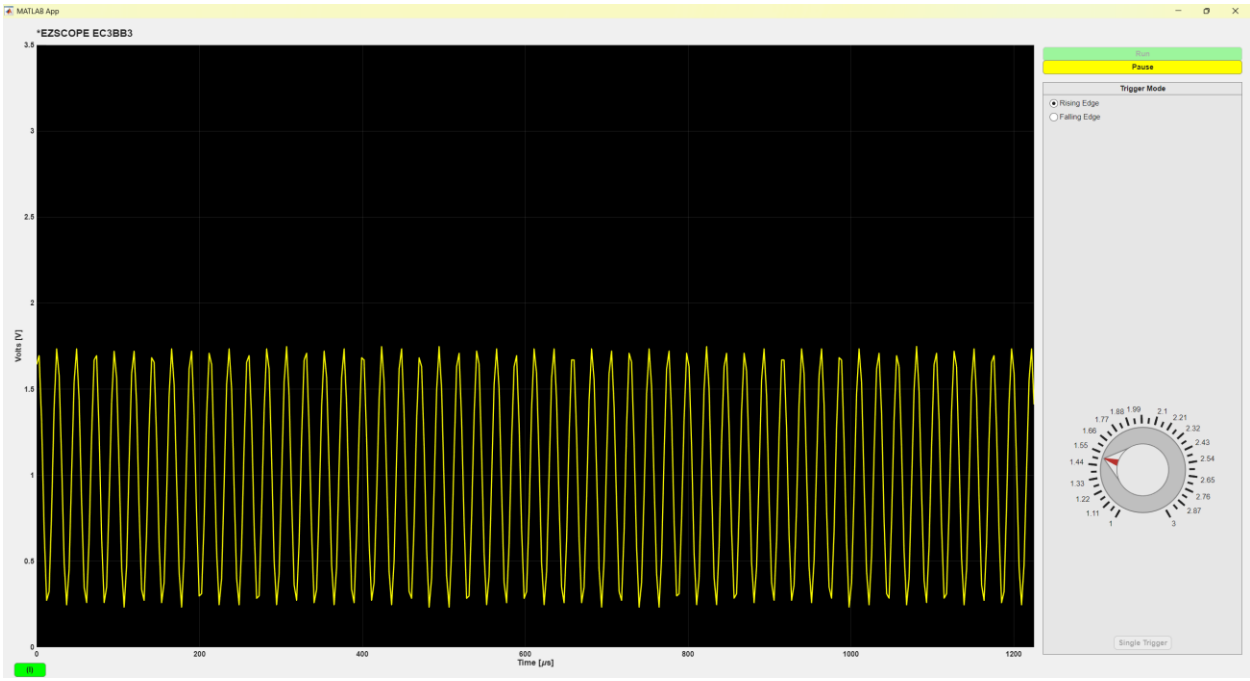

Figure 33: EZSCOPE 30 kHz Sample

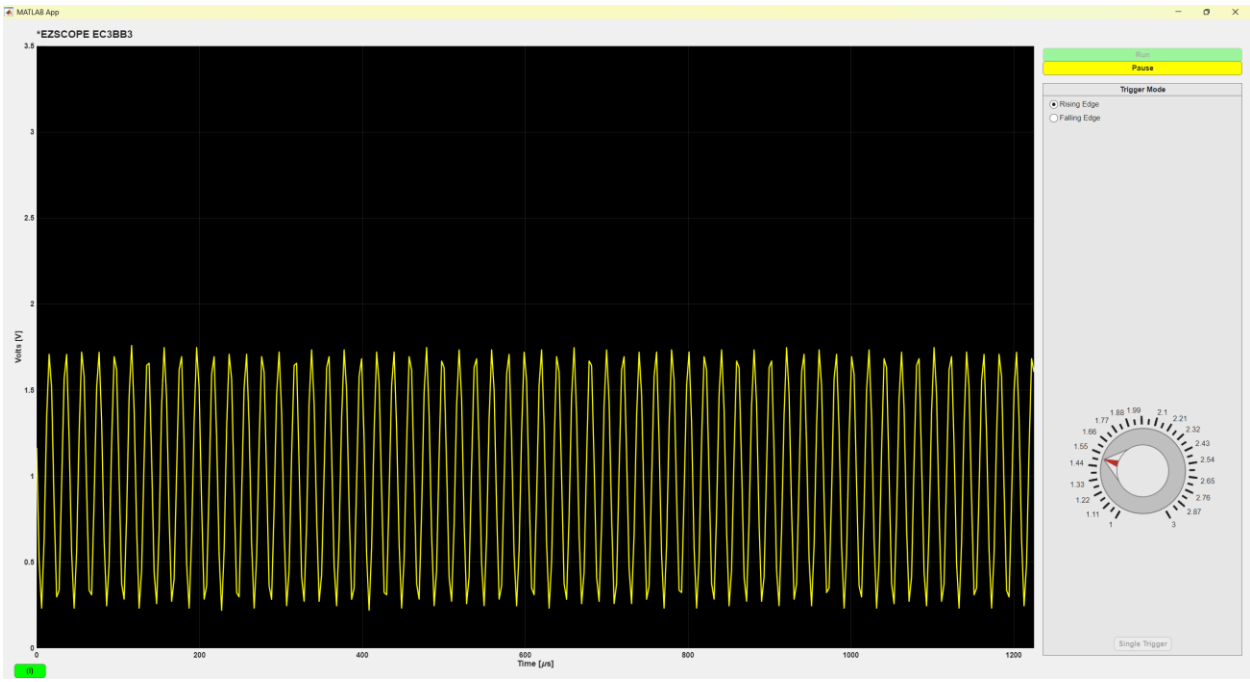

Figure 34: EZSCOPE 35 kHz Sample

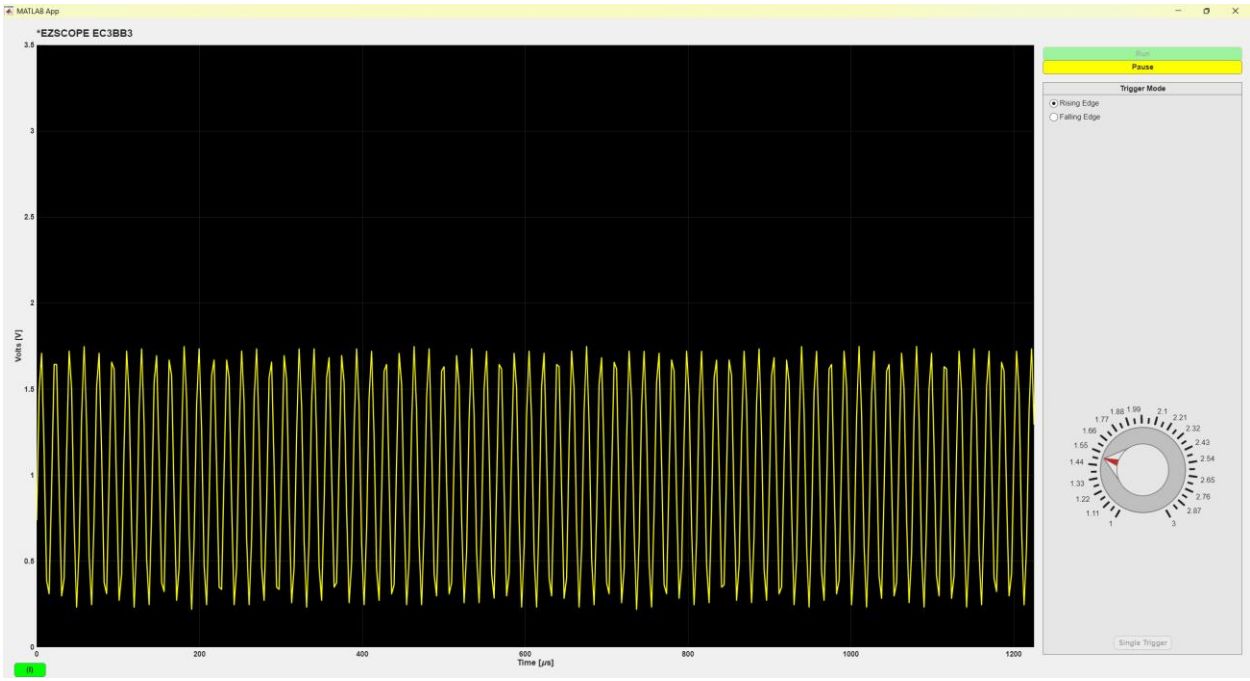

Figure 35: EZSCOPE 40 kHz Sample

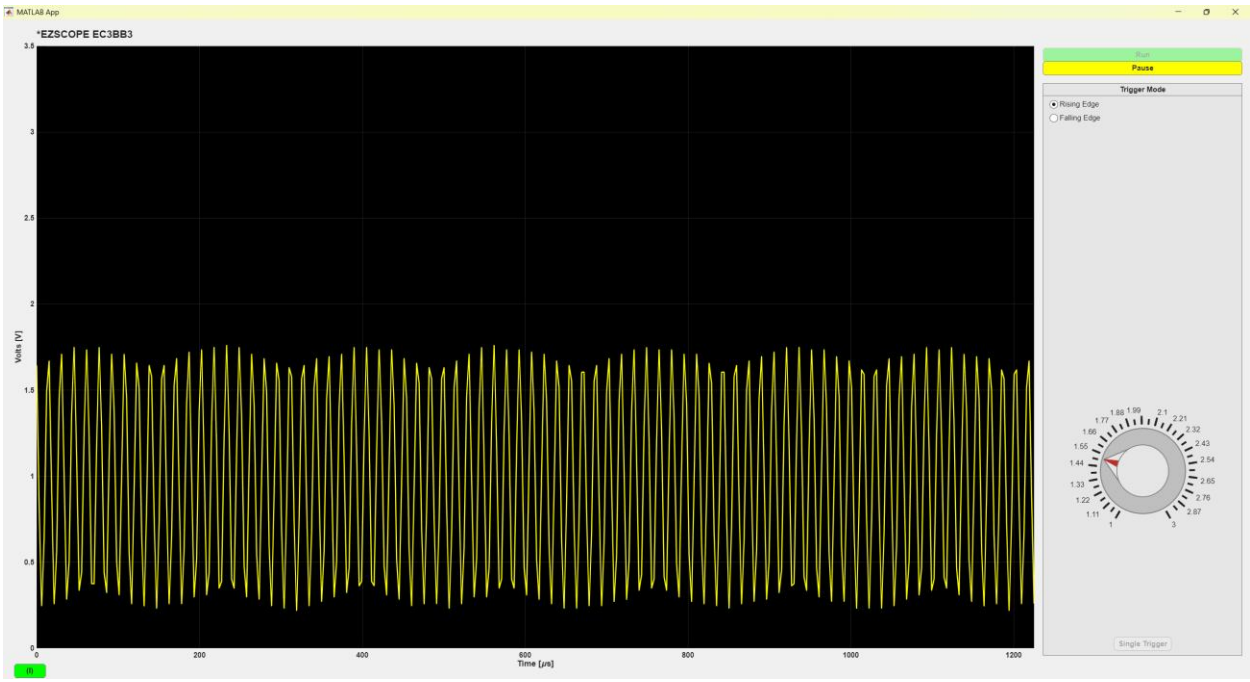

Figure 36: EZSCOPE 45kHz Sample

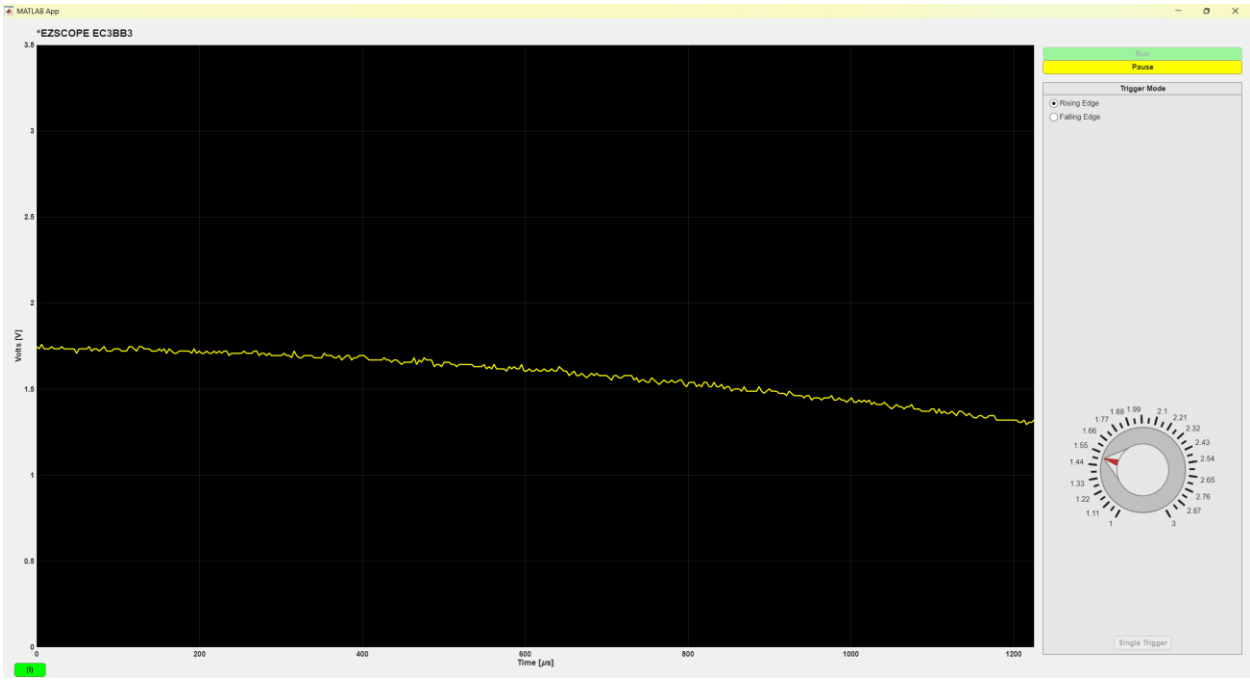

Figure 37: EZSCOPE 100 Hz Sample

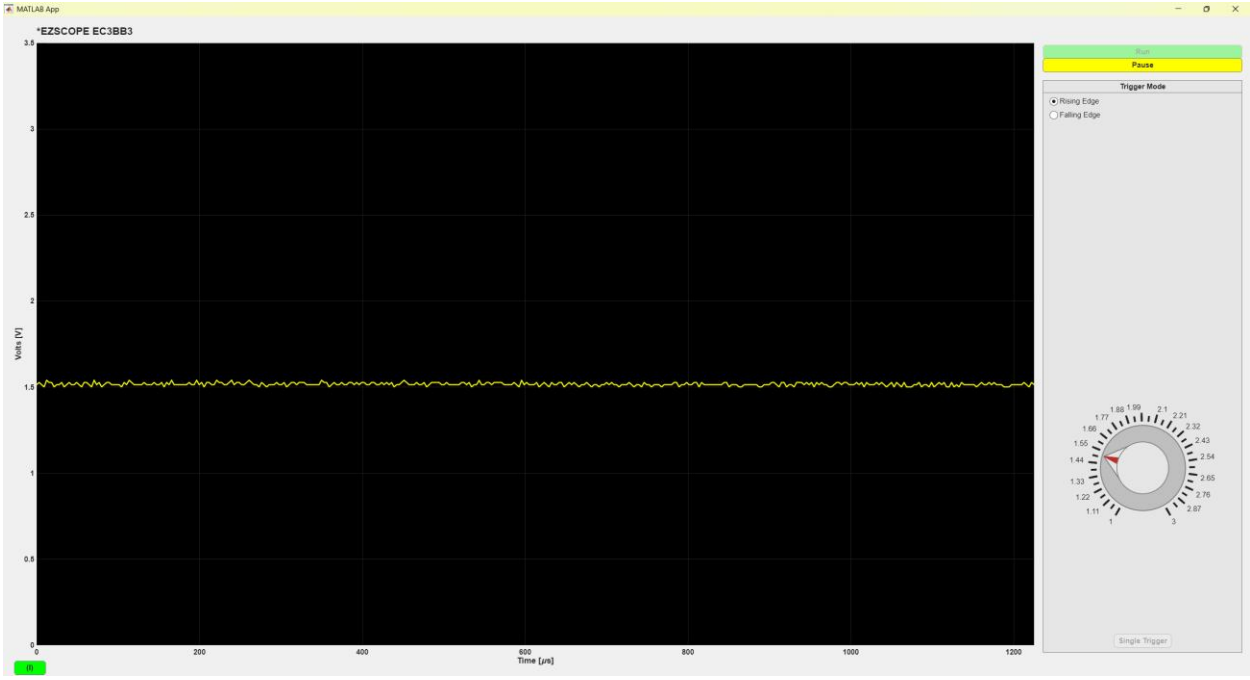

Figure 38: EZSCOPE 1 Hz Sample

References

[1] McMaster University, "EP3BB3 Lab #5-2025/26-V1.2 Lab 5: ADC Module and Serial Interface to MATLAB," McMaster University, Hamilton, ON, Canada, 2026. (accessed February 27, 2026)

[2] Texas Instruments, *MSP430x5xx and MSP430x6xx Family User's Guide*, Rev. F. Dallas, TX, USA: Texas Instruments, Aug. 2012 (Revised Mar. 2018). [Online]. Available: https://www.ti.com/lit/ug/slau315f/slau315f.pdf?ts=1772217144728

[3] GeeksforGeeks, "Nyquist Sampling Theorem," *GeeksforGeeks*, [Online]. Available: https://www.geeksforgeeks.org/electronics-engineering/nyquist-sampling-theorem/